

ПЛОВДИВСКИ УНИВЕРСИТЕТ

„ПАИСИЙ ХИЛЕНДАРСКИ“

Факултет по Математика и Информатика
Катедра „Компютърни Технологии“
Специалност „Софтуерно инженерство и технологии“

ДИПЛОМНА РАБОТА

Тема

Мобилно приложение за по-добра организираност
чрез бележки – Flutter Notes

Дипломант:
Емил Григоров Медаров
Ф№: 2501777012

Научен ръководител:
инф. Виктор Матански

Пловдив, 2026

Съдържание

ДИПЛОМНА РАБОТА.....	1
Списък на фигури.....	3
Списък на таблици	5
Увод	6
Въведение	6
Актуалност	7
Цели и задачи на разработката.....	7
Структура на дипломната работа.....	9
Глава първа – Актуалност на проблема	10
Глава втора – Използвани технологии	20
2.1. Flutter.....	20
2.2. Dart	20
2.3. Firebase	21
2.4. Cloud Firestore.....	22
2.4.1. NoSQL.....	23
Глава трета – Проектиране и функционалност на платформата	25
3.1. Нива на достъп на приложението.....	25
3.2. Структура и функционалност на приложението	26
3.2.1. Страница „Register“	27
3.2.2. Страница „Sign In“	28
3.2.3. Филтриране на бележките по дата на промяна	29
3.2.4. Филтриране на бележките по дата на създаване	30
3.2.5. Промяна на изгледа на бележките	31

3.2.6. Изтриване на бележка	32
3.2.7. Добавяне на бележка.....	33
3.3. Функционалности за форматиране и стилизиране на бележките	35
3.4. UML Диаграми на приложението	57
3.5. Описание на дизайна	61
3.5.1. Избор на цветове	61
Глава четвърта – Реализация и програмен код.....	64
4.1. Реализация на базата данни	64
4.2. Програмна реализация	66
4.2.1. Код за сървиз на бележка.....	66
4.2.2. Код за страница за нова бележка или за редактиране на такава	67
4.2.3. Код за контролер за нова бележка.....	72
4.2.4. Код за страница за възстановяване на паролата	74
Заклучение и постигнати цели	77
Перспективи за бъдещо развитие.....	81
Използвана литература	82

Списък на фигури

Фигура 1. Начална страница на приложението „Бележки в Кеер“	12
Фигура 2. Менюто с функционалностите за всяка бележка	13
Фигура 3. Смяна на цвят на текста и фон на „Бележки в Кеер“ ...	14
Фигура 4. Начална страница на приложението „Mi Notes“	15
Фигура 5. Категоризацията по папки на апликацията „Mi Notes“.	16

Фигура 6. Смяна на фона на бележката на апликацията „Mi Notes“	17
Фигура 7. Менюто с функционалностите на бележките на „Mi Notes“	18
Фигура 8. Начална страница на приложението „Flutter Notes“	27
Фигура 9. Страница за регистрация на апликацията „Flutter Notes“	28
Фигура 10. Страница за вписване в приложението на „Flutter Notes“	29
Фигура 11. Филтриране по дата на промяна на бележката	30
Фигура 12. Филтриране по дата на създаване на бележката	31
Фигура 13. Промяна на изгледа на бележките	32
Фигура 14. Изтриване на бележка	33
Фигура 15. Добавяне на бележка	34
Фигура 16. Добавяне на таг към бележка	35
Фигура 17. Функция „получер“	36
Фигура 18. Функция „курсив“	37
Фигура 19. Функция „подчертан текст“	38
Фигура 20. Функция „зачертан текст“	39
Фигура 21. Функция за смяна на цвета на текста	40
Фигура 22. Функция за смяна на фона на текста	41
Фигура 23. Създаване на списък тип „Арабски цифри“	42
Фигура 24. Създаване на списък тип „Кръг“	43
Фигура 25. Краен резултат с възможност за редактиране на бележката	44
Фигура 26. Търсачка за съвпадения в текста на бележката	45
Фигура 27. Функционалностите за Cut, Copy, Paste и Select All	46
Фигура 28. Функционалността за tag suggestion с Изкуствен интелект	47

Фигура 29. Как работи функцията за <i>tag suggestion</i>	48
Фигура 30. Функцията за добавяне на изображение в бележка	49
Фигура 31. Добавено изображение, което е и в смален размер.....	50
Фигура 32. Диалоговия прозорец при натискане на изображението	51
Фигура 33. Примерна бележка	52
Фигура 34. Това е добавения нов таб.....	53
Фигура 35. Това е резултата от натискането на бутона “New Category”	54
Фигура 36. Тук е показан раздела „Потребителски настройки“....	55
Фигура 37. Това е изгледа на началната страница при включен Тъмен режим	56
Фигура 38. UML Use Case Диаграма, визуализираща цялостният процес в приложението	59
Фигура 39. UML Activity Диаграма, описваща добавянето на нова бележка	60
Фигура 40. Цветно колело	62
Фигура 41. Схема на базата данни на приложението	65

Списък на таблици

Таблица 1. Постигнати резултати от Цел 1	77
Таблица 2. Постигнати резултати от Цел 2	79
Таблица 3. Постигнати резултати от Цел 3	79
Таблица 4. Постигнати резултати от Цел 4	80

Увод

Въведение

В днешното динамично ежедневие технологиите играят ключова роля в живота на хората. Те улесняват, ускоряват и подобряват комуникацията, достъпа до информация и личната организация. Повечето потребители разчитат на смартфони и мобилни приложения, за да записват бързи идеи, да планират задачи и да поддържат ред в своите ангажименти. В този контекст приложенията за бележки се оказват изключително полезни, тъй като предлагат гъвкавост и достъпност във всяка ситуация.

Основната цел на едно мобилно приложение за бележки е да осигури на потребителите ясна, интуитивна и ефективна среда за записване, съхранение и управление на информация. За да се постигне това, е необходимо първо да се идентифицират различните типове потребители – студенти, професионалисти, хора с натоварено ежедневие – и техните нужди от бърз достъп, категоризиране и синхронизация на бележките.

Настоящата дипломна работа, озаглавена „Flutter Notes“, демонстрира създаването и разработката на мобилно приложение за бележки и по-добра организираност. В нея са разгледани всички етапи от проектирането и имплементацията на системата, както и технологиите, използвани за изграждане на бизнес логиката и потребителския интерфейс с платформата Flutter. Приложението е създадено, за да отговори на изискванията за бързодействие, интуитивна навигация, синхронизация и сигурност, като крайният софтуерен продукт напълно покрива нуждите на хората, търсещи по-добра лична организация.

Актуалност

В рамките на дипломната работа ще бъдат анализирани някои от най-съществените аспекти на мобилното приложение, сред които неговата функционалност и начините, по които подпомага потребители от различни области. В интернет съществуват множество приложения от този вид. След извършване на подробно проучване и анализ беше направен изводът, че настоящата разработка не прилича на всички малки и непопулярни приложения, съществуващи към момента, което ѝ придава уникалност и я прави важна за усъвършенстването на процеса на по-добра организация от страна на потребителя.

Основната цел на приложението е да предложи на потребителите възможност за създаване на бележки чрез разнообразни функции, като добавяне на кратки описателни тагове, използване на получер и курсив стил, оформяне на списъци в бележката, както и опция за търсене на съвпадения в текста. Приложението се отличава с лесен достъп и интуитивен интерфейс, които гарантират отлично потребителско изживяване и пълноценно използване на предлаганите възможности. В съвременното почти всеки човек разчита на приложение за бележки, за да изпълнява задълженията си по-ефективно. Настоящото приложение съчетава няколко идеи за функционалности, характерни за подобен тип продукти, които са мое лично виждане и са ми хрумнали след разглеждането на аналогични приложения.

Цели и задачи на разработката

Настоящата дипломна работа под надслов "Мобилно приложение за по-добра организираност чрез бележки – Flutter Notes" си поставя за цел да реализира няколко значими и съвременни задачи, чието успешно изпълнение допринася за разрешаването на комплексния проблем в неговата цялост.

Цел 1: Да бъде създаден завършен софтуерен продукт, който да представлява универсално средство за хора, стремящи се към по-добра организираност и повишена ефективност при изпълнение на ежедневните си задачи.

От тази цел произтичат следните подзадачи:

- Да се осигури интуитивен интерфейс, гарантиращ лека и бърза навигация в приложението;
- Да се предостави възможност за изграждане на потребителски профил и вписване в системата;

- Да се реализира функционалност за добавяне на бележки, както и за прилагане на допълнителни опции и стилове;
- Да се създадат повече механизми за сортиране по дата на последна промяна и дата на създаване, както и опция за промяна на визуалния изглед на бележките.

Цел 2: Да се осъществи анализ и сравнение на вече съществуващи платформи.

От тази цел произтичат следните подзадачи:

- Да бъдат проучени най-популярните платформи, свързани с разглежданата проблематика;
- Да се анализира достъпността на тези платформи, включваща преглед на техния дизайн и структура;
- Да се акцентира върху разнообразните функционалности на платформите и начина, по който те се различават от останалите.

Цел 3: Да бъде изграден краен софтуерен продукт, който да удовлетворява поставените цели, свързани с добавяне на записки, осигуряване на стабилна функционалност на потребителя на апликацията.

От тази цел произтичат следните подзадачи:

- Да бъде разработена система с добра функционалност на потребителя, която да осигурява актуалност на данните, лесна организираност и бързи реакции;
- Да се проектира система за въвеждане на записки с множество полезни функции за потребителите в реално време, която да гарантира актуалност на данните и лесна организираност;
- Потребителят да може да добавя бележки, да ги изтрива, да ги редактира и да разполага с достъп до всички налични опции, които са вградени в приложението.

Цел 4: При разработката на мобилната платформа да бъдат използвани съвременни методи и технологии за мобилно програмиране.

От тази цел произтичат следните подзадачи:

- Да бъдат представени технологиите, приложени при изграждането на конкретния софтуерен продукт;
- Да се предоставят подробни сведения относно различните компоненти и използвания програмен код.

Структура на дипломната работа

Разработката има за цел да представи технологичните подходи, които ще бъдат приложени при изграждането на платформата „Flutter Notes“, както и да разреши формулирания в дипломната работа проблем, като структурата ѝ е разделена на няколко основни части.

В *Глава първа, озаглавена „Актуалност на проблема“*, се анализират съществуващите и някои от най-популярните платформи в интернет пространството, като се акцентира върху наличието на подобни приложения. Описва се глобалният проблем и недостатъчното предлагане на платформа, която да задоволи потребностите на хората за водене на бележки на малък екран, тъй като това вече е по-удобно и съвременно в сравнение с писмената форма на хартия. Целта на тази глава е да предостави контекст за проблема и неговата значимост в обществото, както и да изпълни цел 2 от дипломната разработка.

В *Глава втора, озаглавена „Използвани технологии“*, се описват подробно технологиите, които ще бъдат приложени за решаване на проблема. Обсъждат се програмните езици и системите за управление на бази данни. Целта на тази глава е да предостави детайли относно всички технологии, използвани по време на разработката на софтуерния продукт, и да реализира цел 4 от дипломната работа.

В *Глава трета, озаглавена „Проектиране и функционалност на платформата“*, се разглеждат функционалните процеси, през които преминава потребителят. Описва се изборът на дизайн и цветови схеми, които са от съществено значение за потребителското изживяване. Разглеждат се възможностите на различните типове потребители – регистрирани и нерегистрирани – и се представят UML диаграми на случаите на употреба за детайлно описание на приложението. Целта на тази глава е да опише всички функционалности на системата и да изпълни цел 1 и цел 3 от дипломната разработка.

В *Глава четвърта, озаглавена „Реализация и програмен код“*, се обяснява как са реализирани отделните компоненти на системата чрез програмен код и как те са свързани помежду си. Включени са диаграми и схема на базата данни. Целта на тази глава е да

предостави подробно описание на програмната реализация на системата и да изпълни цел 4 от дипломната работа.

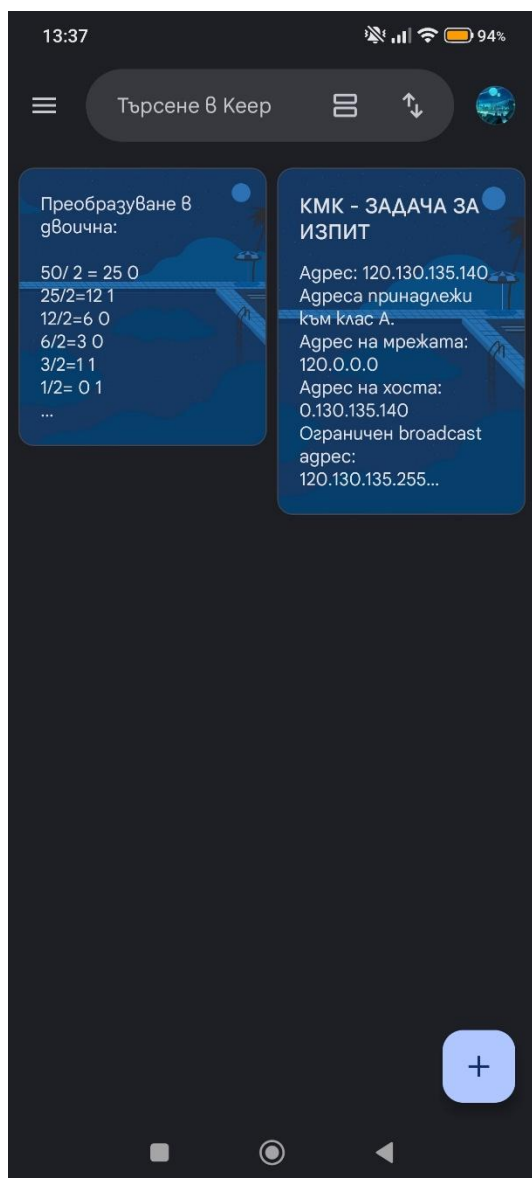
Глава първа – Актуалност на проблема

В хода на подготовката на настоящата дипломна работа бяха разгледани множество софтуерни реализации със сходна насоченост в интернет. Анализът имаше за задача да изследва съответните мобилно базирани решения, за да се постигне по-голяма яснота относно дефинирането на основните функционалности, които предстои да бъдат разработени. Основните критерии за сравнение на съществуващите решения, които повлияха върху оформянето на концепцията на настоящата дипломна работа, са следните:

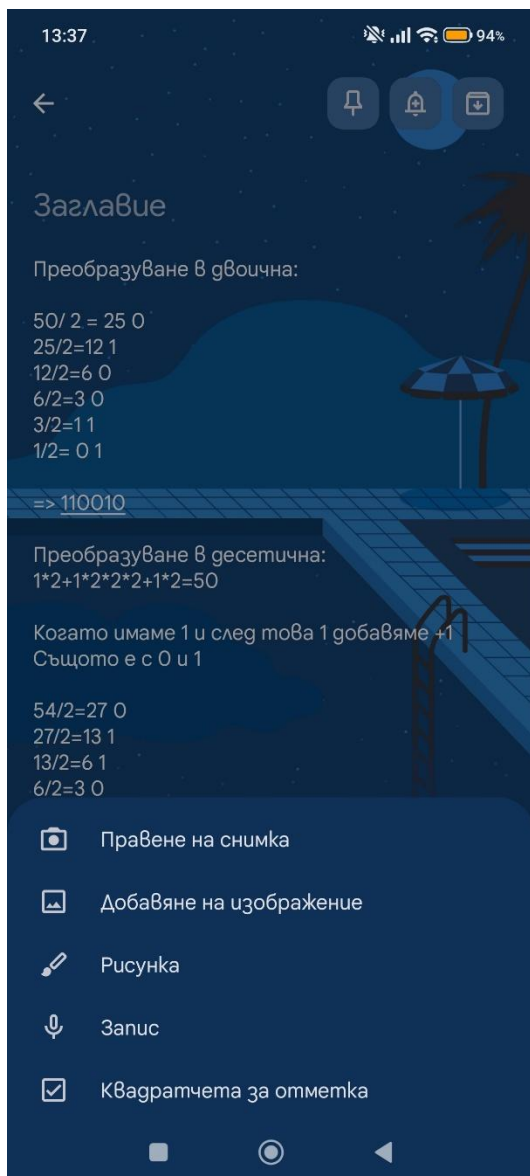
- Целева група – установяване на потребителите, за които даденото приложение би било полезно (кой би го посетил, разгледал и използвал).
- Графичен потребителски интерфейс (дизайн) – преценка доколко дизайнът е интуитивен, удобен и привлекателен за потребителите, дали е съобразен с целевата аудитория, както и подходящостта на цветовата гама спрямо представената информация.
- Навигация (достъпност на приложението) – анализ на това колко лесно обикновеният потребител може да се ориентира в различните екрани и съдържанието на приложението.
- Съдържание (значимост и структурираност на информацията) – преглед на съдържанието на приложението, неговата релевантност към темата и начинът, по който информацията е организирана и представена.
- Потребителска оценка – събиране на мнения от ползвателите за съответното приложение, сравнение на силните страни (какво го отличава от другите със сходна тематика) и слабите страни (какво биха променили потребителите).

Въз основа на проведеното изследване на съществуващите решения в интернет пространството беше постигната значителна яснота относно основните изисквания, които настоящото мобилно приложение трябва да покрие, за да обслужва ефективно и адекватно потребностите на своите потенциални ползватели.

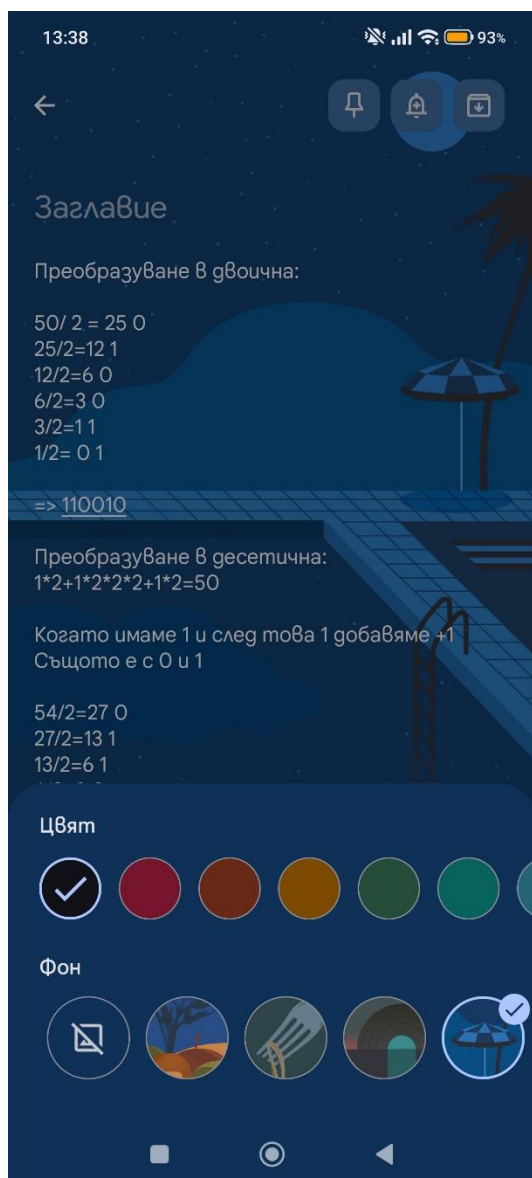
На първо място беше разгледано приложението на Google – „Бележки в Кеер“, представено на фигура 1. То е най-близко до желания софтуерен продукт, който следва да бъде разработен за целите на тази дипломна работа. Това е най-популярното приложение за бележки в Google Play Store по отношение на разглежданата тематика в научната разработка. Неговите потребители са предимно хора, които желаят да бъдат организирани, интелигентни и информирани, както и организации. Дизайнът на началния екран е до известна степен интуитивен, но посетителят трябва да е запознат с целите на организацията още преди посещението. Налице са светъл и тъмен режим, които са подходящи и създават усещане за приятно изживяване. Информацията е представена по полезен начин. Отличителна характеристика на платформата е възможността за добавяне на изображение, рисунка или звуков запис към бележката, като там се намира менюто с функционалностите на всяка бележка (фигура 2). Освен това може да се променят цветът на бележката и фонът (фигура 3).



Фигура 1. Начална страница на приложението „Бележки в Кеер“



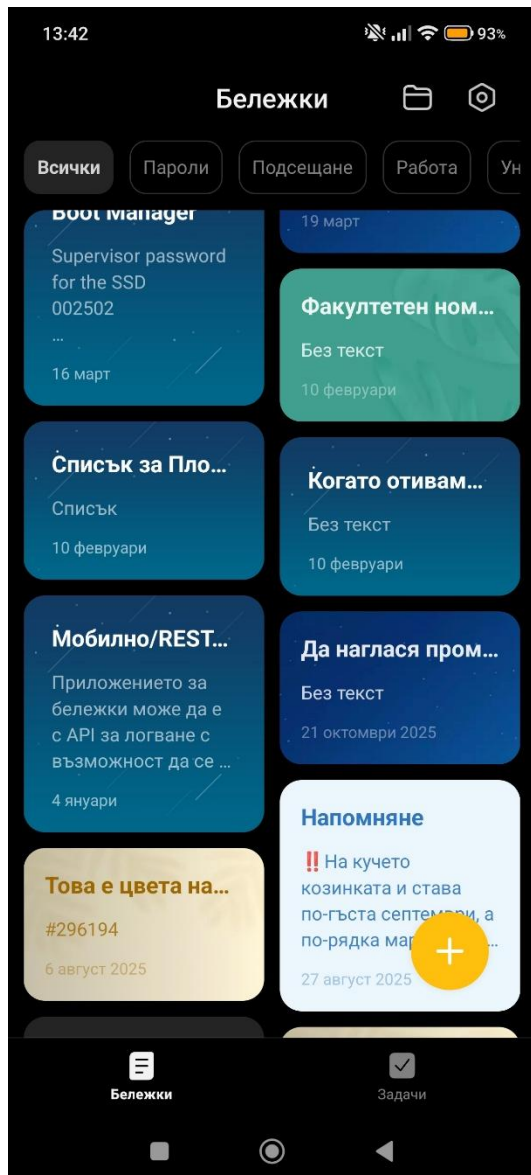
Фигура 2. Менюто с функционалностите за всяка бележка



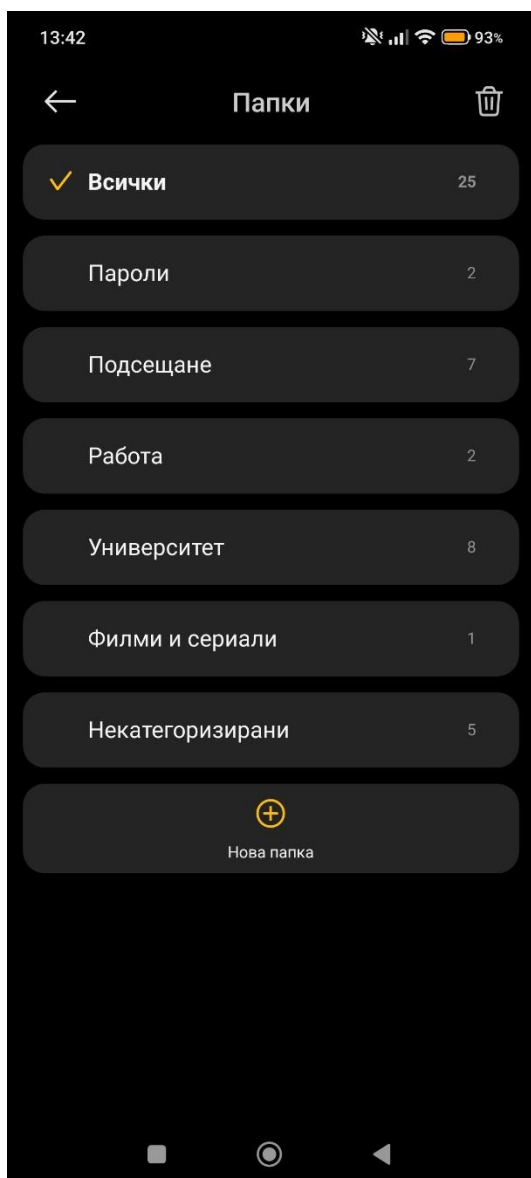
Фигура 3. Смяна на цвят на текста и фон на „Бележки в Кеер“

Второто по популярност приложение, свързано с разглежданата тематика и включено в анализа, е Mi Notes на производителя на техника Xiaomi (фигура 4). Целевата група отново обхваща хора, които желаят да бъдат организирани, интелигентни и подредени, както и различни организации. Дизайнът тук е опростен и удобен, с навигационно меню, което използва множество икони за по-лесно възприемане на информацията от потребителите. Отличителна черта на тази платформа е наличието на повече функционалности, включително опция за групиране на бележките в папки. Наличието на тъмен режим отново създава усещане за радост и удоволствие. Информацията е представена по лесен и достъпен начин. Това приложение, разбира се, е достъпно само за устройства на Xiaomi и се предлага в AppMall. Отличителни характеристики на

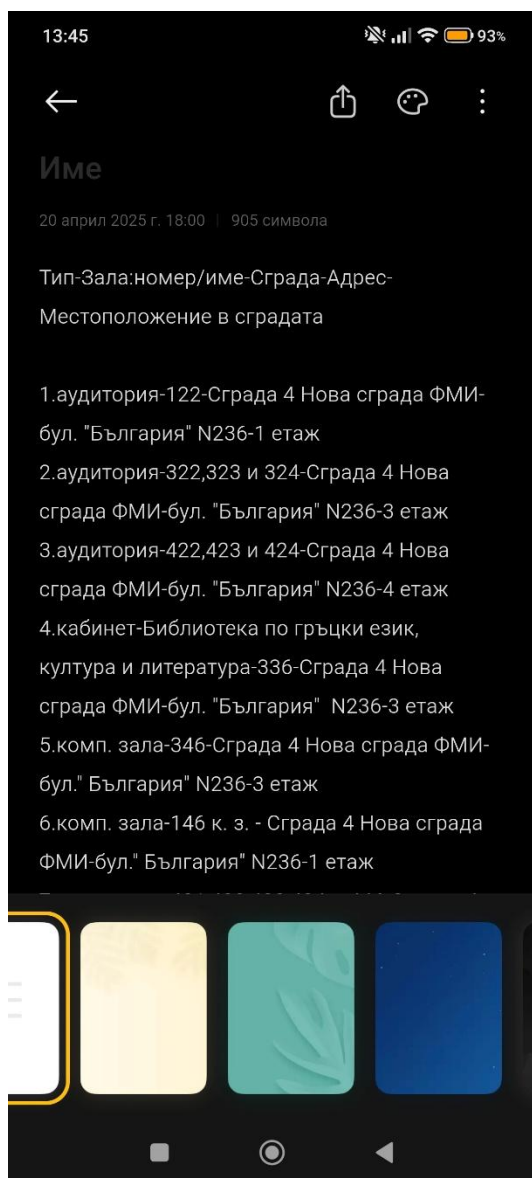
платформата са възможността за подреждане на бележките в папки (фигура 5), смяна на фона на бележката (фигура 6), както и добавяне на изображение, рисунка или звуков запис към бележката, като там се намира менюто с функционалностите на всяка бележка (фигура 7).



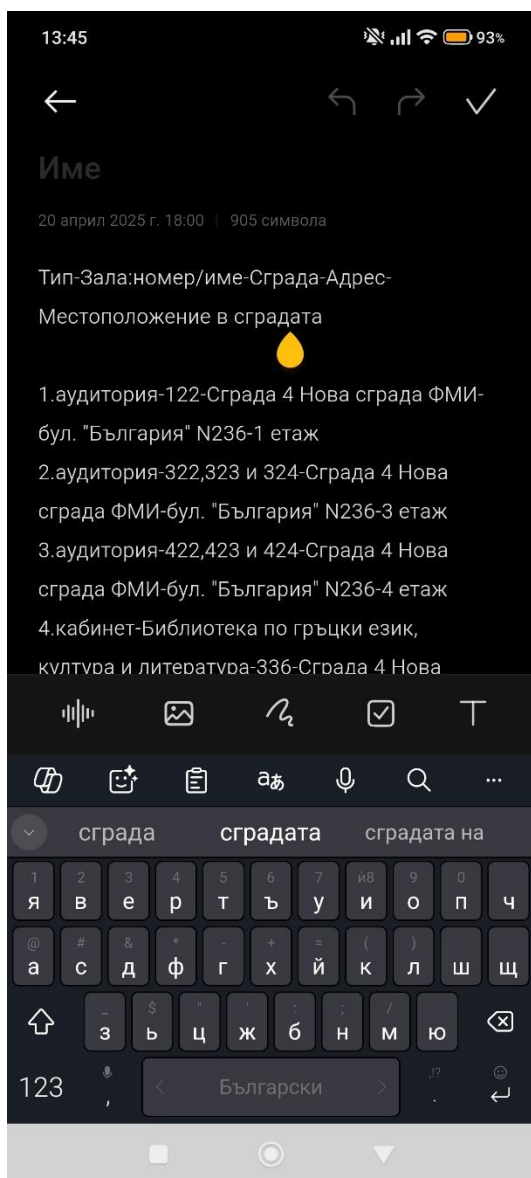
Фигура 4. Начална страница на приложението „Mi Notes“



Фигура 5. Категоризацията по папки на апликацията „Mi Notes“



Фигура 6. Смяна на фона на бележката на апликацията „Mi Notes“



Фигура 7. Менюто с функционалностите на бележките на „Mi Notes“

Магазините за приложения също играят важна роля в анализа на разглеждания проблем. Тъй като приложенията със сходна тематика са изключително разпространени в интернет, потребителите често тестват и експериментират с различни от тях, докато открият подходящо в Google Play Store или друг подобен магазин за приложения. За нуждите на анализа бяха разгледани различни приложения, които преследват идентична цел.

Въз основа на извършения анализ може да се направи изводът, че обединяването на различните подсистеми и техните функционалности в една нова система, наречена „Мобилно приложение за по-добра организираност чрез бележки – Flutter

Notes“, ще допринесе за подобряване на процеса на организираност. Това подчертава значимостта на целите на дипломната работа, която разглежда актуален проблем и предлага цялостно решение за него.

Въпреки че на пазара съществуват множество приложения за бележки, „Flutter Notes“ се отличава с редица функционалности, които го правят по-практично и по-добре приспособено към нуждите на съвременния потребител.

Приложението разполага с функция за интелигентно предлагане на тагове чрез Изкуствен интелект.

Една от най-значимите отличителни черти на приложението е интегрираната функционалност за автоматично предлагане на тагове. Чрез приложния програмен интерфейс на големия езиков модел Llama на Meta (достъпен през Cloudflare Workers AI), приложението анализира съдържанието на всяка бележка поотделно и предлага подходящ таг. Тази възможност не се среща често в масовите приложения за бележки и спестява време на потребителя при категоризирането на информацията.

За разлика от много приложения, които предлагат само базово форматиране, „Flutter Notes“ предоставя богат набор от разширени възможности за форматиране и стилизиране.

Приложението предлага и гъвкаво структуриране чрез категории, тагове и приоритети, като също и в него може да се сортира и променя изгледа, може още да се прикачват и изображения и създават напомняния, предлага също и потребителски настройки.

Дизайнът на приложението е опростен и функционален, с ясна навигация и последователна цветова палитра от жълто, бяло, черно и сиво. Тази комбинация създава баланс между топлина и прегледност и улеснява фокусирането върху съдържанието.

В обобщение, „Flutter Notes“ съчетава в едно приложение функционалности, които обикновено са разпръснати в различни решения – интелигентно тагване чрез изкуствен интелект, богато форматиране, многостепенна организация, прикачване на файлове, напомняния, синхронизация и офлайн работа. Именно това го прави по-универсално и по-практично от голяма част от съществуващите приложения за бележки.

Глава втора – Използвани технологии

Въз основа на формулираните цели и направения анализ на съществуващите решения, може да се премине към описание на технологиите, използвани за разработката на системата. Описанието на тези технологии е извлечено от съответните им документации, които са достъпни в интернет.

2.1. Flutter

Flutter е Google комплект за разработка с отворен код за създаване на междуплатформени приложения от една кодова база за Android, iOS, уеб, Windows, macOS и Linux. Първоначално описан през 2015 г., Flutter е официално пуснат през май 2017 г. Той се използва в продукти като Google Pay и Google Earth.

За разлика от други рамки, които разчитат на native компоненти на целевата платформа, Flutter разполага със собствен рендиращ енджин, който директно изобразява пиксели на екрана. Ключова характеристика е горещото презареждане, позволяващо мигновено виждане на промените в кода без рестартиране на приложението.

Основен градивен елемент във Flutter е виджетът, който може да съдържа други виджети. Съществуват два вида виджети: без състояние и със състояние. Първите се актуализират само при промяна на входните данни, докато вторите могат да извикват `setState()`, за да обновят вътрешното си състояние и да се преназначат.

Flutter включва два набора от виджети – Material Design (на Google) и Cupertino (на Apple за iOS), като и двата могат да се използват на всяка платформа.

Приложенията се пишат на езика Dart. За всички платформи освен уеб се използва предварителна компилация (AOT), а за уеб кодът се трансформира до JavaScript или WebAssembly. Мениджърът на пакети Pub и хранилището на Dart позволяват споделяне на персонализирани пакети и плъгини.

Двигателят на Flutter е написан основно на C++ и осигурява ниско ниво на рендиране чрез графичната библиотека Skia или Impeller (активен по подразбиране за iOS и Android API 29+). Той комуникира с платформено-специфичните SDK за достъп до файловата система, мрежата, достъпността и други.

2.2. Dart

Dart е обектно-ориентиран език за програмиране, разработен от Google, със синтаксис в стил C. Той поддържа интерфейси, абстрактни класове, генерични типове и извеждане на типове.

Използва се за изграждане на мобилни, уеб, сървърни и настолни приложения, като може да се компилира до машинен код, JavaScript или WebAssembly.

Dart е представен през октомври 2011 г., а версия 1.0 излиза през ноември 2013 г. Първоначално приемът е смесен, но през 2015 г. Google се отказва от плановете за вграждане на Dart VM в Chrome и фокусът се насочва към компилация до JavaScript.

С версия 2.0 (август 2018 г.) са въведени промени в типовата система. Версия 2.6 (ноември 2019 г.) въвежда dart2native, позволявайки компилация до самостоятелни изпълними файлове за Linux, macOS и Windows без нужда от инсталиран Dart SDK. Flutter инструментариумът интегрира Dart за компилация и backend поддръжка.

Dart 3.0 (май 2023 г.) въвежда надеждна null безопасност, както и записи, шаблони и модификатори на класове. От версия 3.4 Dart поддържа компилация до WebAssembly.

Dart е стандартизиран от ECMA чрез технически комитет TC52, като първата версия на спецификацията (ECMA-408) е приета през юли 2014 г.

Синтаксисът на каскадата от методи е вдъхновен от Smalltalk, а миксините – от Strongtalk и Ruby. Dart използва изолати (базирани на актьорския модел от Erlang) за паралелизъм и сигурност. Концепцията за Mirror API за контролирано отражение е реализирана първо в Self.

2.3. Firebase

Firebase е създадена като компания през 2011 г. в Сан Франциско, а през 2014 г. е придобита от Google. Днес Firebase представлява набор от облачни backend услуги, включващи бази данни, удостоверяване на потребители, хостинг и интеграция за платформи като Android, iOS, JavaScript, Node.js, Java, Unity, PHP и C++.

Първоначално Firebase произлиза от проекта Envolvе, който предлага онлайн чат функционалност. Поради неочакваната употреба на услугата за синхронизация на данни в реално време (например състояние на игра), основателите Джеймс Тамплин и Андрю Лий отделят чат системата и създават Firebase. През април 2012 г. е пуснат първият продукт – Firebase Realtime Database.

През май 2012 г. Firebase набира 1,1 млн. долара начално финансиране, а през юни 2013 г. – още 5,6 млн. долара. През 2014 г.

добавя Firebase Hosting и Firebase Authentication, превръщайки се в платформа тип "backend-as-a-service" за мобилни приложения.

След придобиването от Google през октомври 2014 г., на Google I/O през май 2016 г. е представен Firebase Analytics, а Firebase Cloud Messaging заменя Google Cloud Messaging с поддръжка за Android, iOS и уеб. През юли 2016 г. Google придобива LaunchKit, а през януари 2017 г. – Fabric и Crashlytics, които са интегрирани във Firebase. През октомври 2017 г. е пуснат Cloud Firestore като наследник на оригиналната Realtime Database.

През 2020 г. е заведено дело срещу Google за проследяване на потребители чрез Firebase без тяхно знание, но делото е отхвърлено през януари 2022 г.

2.4. Cloud Firestore

Можете да използвате наред с Firebase и гъвкавата и мащабируема NoSQL облачна база данни (Cloud Firestore), изградена върху инфраструктурата на Google Cloud, за да съхранявате и синхронизирате данни при клиентска и сървърна разработка.

Cloud Firestore е гъвкава, мащабируема база данни за мобилна, уеб и сървърна разработка, част от Firebase и Google Cloud. Подобно на Firebase Realtime Database, тя синхронизира данните в клиентските приложения чрез слушатели в реално време и предлага офлайн поддръжка за мобилни устройства и уеб, което позволява изграждането на адаптивни приложения, работещи независимо от мрежовата латентност или интернет връзката. Cloud Firestore осигурява безпроблемна интеграция с други продукти на Firebase и Google Cloud, включително Cloud Run функциите.

Cloud Firestore предлага две издания – Standard и Enterprise – за да отговори на различни нужди. Това е NoSQL база данни, хоствана в облак, с директен достъп от Apple, Android и уеб приложения чрез родни SDK. Налична е също така с Node.js, Java, Python, Unity, C++ и Go SDK, както и чрез REST и RPC API.

Моделът на данните на Cloud Firestore следва документно-ориентиран подход – съхранявате документи с полета и съответстващи им стойности. Документите се групират в колекции, които улесняват организацията на данните и изграждането на

заявки. Документите поддържат различни типове данни – от прости низове и числа до сложни вложени обекти. Можете да създавате подколекции в документите, изграждайки йерархични структури, които се мащабират с растежа на базата данни. Този модел поддържа всяка структура от данни, подходяща за вашето приложение.

Заявките в Cloud Firestore са изразителни, ефективни и гъвкави. Създавайте плитки заявки за извличане на данни само на ниво документ, без да зареждате цялата колекция или вложените подколекции. Добавяйте сортиране, филтри, ограничения и курсори за номериране на резултатите. За поддържане на актуалност на данните без пълно повторно извличане при всяка актуализация, добавяйте слушатели в реално време. Те уведомяват приложението с моментна снимка на данните при промяна, извличайки само новите разлики.

В Cloud Firestore можете да защитите достъпа до данните чрез Firebase Authentication и правила за сигурност (Security Rules) за Android, Apple платформи и JavaScript, или чрез Identity and Access Management (IAM) за сървърни езици.

2.4.1. NoSQL

В днешно време информацията е навсякъде и постоянно заобикаля хората. Тя е изключително ценен ресурс, който създава приходи и работа, но също така предизвиква проблеми. Информацията се търгува, краде и използва за различни цели, като почти никой не може да я пренебрегне. Поради това базите данни се налагат като най-ефективното средство за съхранение, управление и обработка на информация. Всяка съвременна компания разчита на данни, които са от решаващо значение за нейното оцеляване, тъй като те са основен фактор при вземането на стратегически решения.

Нерелационната база данни (NoSQL – от английски „Not only SQL“) предоставя механизъм за съхранение и извличане на данни, който използва свободен модел на съгласуваност, за разлика от разпространената релационна база данни. Ползите от този подход включват хоризонтално мащабиране. Нерелационната база данни обикновено представлява добре оптимизирано хранилище от тип ключ-стойност. Нейната цел е да улесни процесите по извличане и добавяне на информация с оглед оптимизиране на производителността. NoSQL намира значително и нарастващо приложение в реалновремевите уеб и big data приложения.

Нерелационните системи често биват наричани „Not only SQL“, за да се подчертае, че те позволяват използването на езици за търсене, различни от SQL, но сходни с него.

Нерелационната база данни притежава разпределена и устойчива на грешки архитектура, като информационните обекти се съхраняват на няколко сървъра. По този начин системата може да се предпази чрез добавяне на повече сървъри и да продължи да функционира при отказ на отделен сървър. Този тип база данни се разширява хоризонтално и се използва за управление на големи обеми от данни, когато производителността (в реално време) е по-важна от последователността – например при индексирани на голям брой документи, обслужване на страници на натоварени уебсайтове или предоставяне на потоци от данни.

ACID срещу BASE. Нерелационните бази данни не могат непременно да дадат пълна гаранция за ACID (атомарност, съгласуваност, изолация, устойчивост). Обикновено съгласуваността се гарантира само частично или транзакциите са ограничени до един информационен обект. Това означава, че при достатъчно дълъг период от време без промени може да се очаква всички обновления да се разпространят в системата.

Терминът „нерелационна база данни“ е използван за първи път от Карло Строци през 1998 г., за да назове неговата лека релационна база данни с отворен код, която не предоставя стандартен SQL интерфейс. Поради отклонението на NoSQL движението от релационния модел, Строци предлага по-подходящото наименование „NoREL“.

Ерик Еванс (тогава служител в Rackspace) възстановява употребата на термина в началото на 2009 г., когато Йоан Оскарсон от музикалния уебсайт Last.fm го моли да организира събитие, посветено на разпределени бази данни с отворен код. Името е използвано като етикет за нововъзникващите и нарастващи по брой нерелационни, разпределени хранилища от данни, които не се стремят да гарантират ACID – ключов атрибут на класическите релационни системи за управление на бази данни.

Глава трета – Проектиране и функционалност на платформата

3.1. Нива на достъп на приложението

Настоящото мобилно приложение за организираност и подобряване на паметливостта чрез бележки е разработено в съответствие с потребностите на масовия интернет потребител. Платформата е надеждна и разполага с удобен и опростен графичен интерфейс, съобразен с комфорта на крайния ползвател.

Целта на приложението е да предостави на всеки, нуждаещ се от платформа за записване и напомняне, лесен и достъпен начин за използване. Въпреки многобройните съществуващи приложения със сходна тематика, малко от тях представят информацията по разбираем и приятен за потребителя начин. Приложението се стреми да подобри процеса на информираност, като предлага разнообразни възможности и функционалности, които повишават неговата полезност и цялостната му взаимовръзка с потребителите.

Приложението дефинира следните потребителски роли:

- Нерегистриран потребител
- Регистриран потребител

Нерегистриран потребител

Нерегистрираният потребител (гост) има достъп до ограничен набор от функции, включително:

- Възможност за регистрация чрез попълване на формата;
- Възможност за регистрация чрез Google или Facebook;
- Възможност за влизане в приложението чрез страницата за вход.

Въпреки това, гостът е ограничен и се нуждае от регистрация, за да получи достъп до допълнителни функционалности.

Гостът НЕ може:

- Да създава или изтрива бележки;

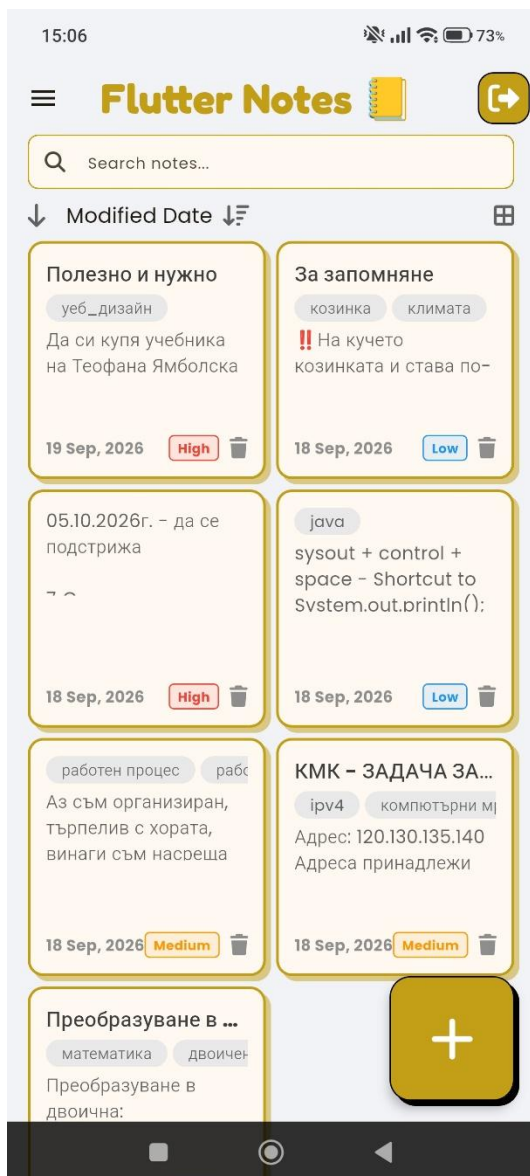
- Да филтрира по дата на създаване или дата на промяна;
- Да променя изгледа на бележките;
- Да използва функциите за редактиране и форматиране на бележки.

С едно щракване върху бутона "Регистрация", всеки гост може да създаде профил, което му предоставя допълнителни възможности като **Регистриран потребител**:

- Създаване на бележки;
- Редактиране и изтриване на собствени бележки;
- Филтриране по дата на създаване и дата на промяна;
- Промяна на изгледа на бележките;
- Възстановяване на забравена парола;
- Достъп до функциите за редактиране и форматиране на бележки.

3.2. Структура и функционалност на приложението

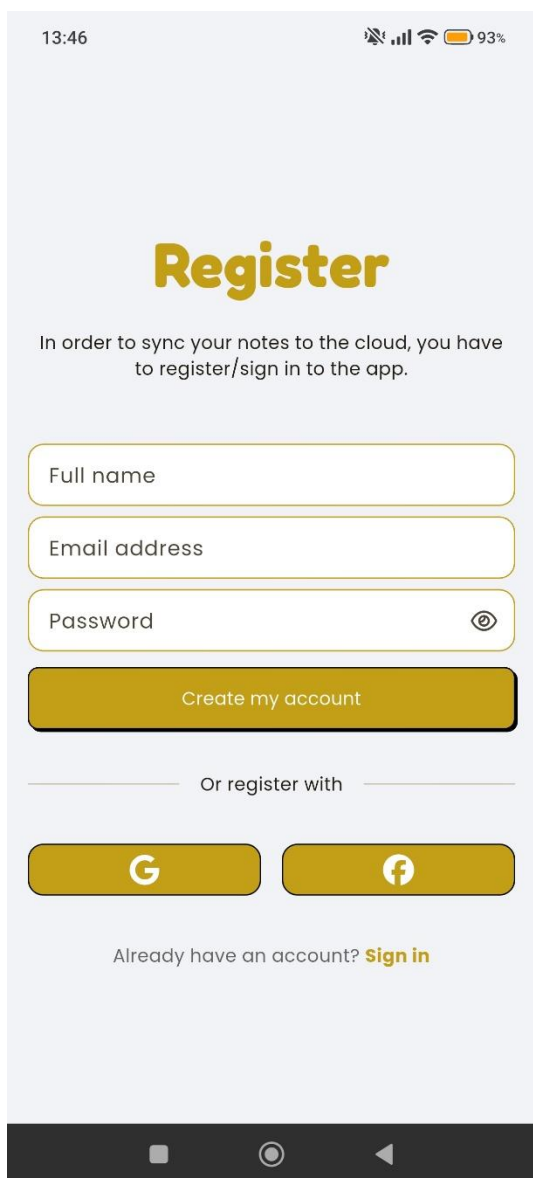
Правилното структуриране на страниците в мобилното приложение улеснява потребителите при намирането на търсеното съдържание. Визуалната организация на приложението се изразява чрез неговата навигация или меню. От тях зависи дали ползвателят ще се ориентира бързо и лесно, както и дали информацията ясно разкрива предназначението на всяка страница.



Фигура 8. Начална страница на приложението „Flutter Notes“

3.2.1. Страница „Register“

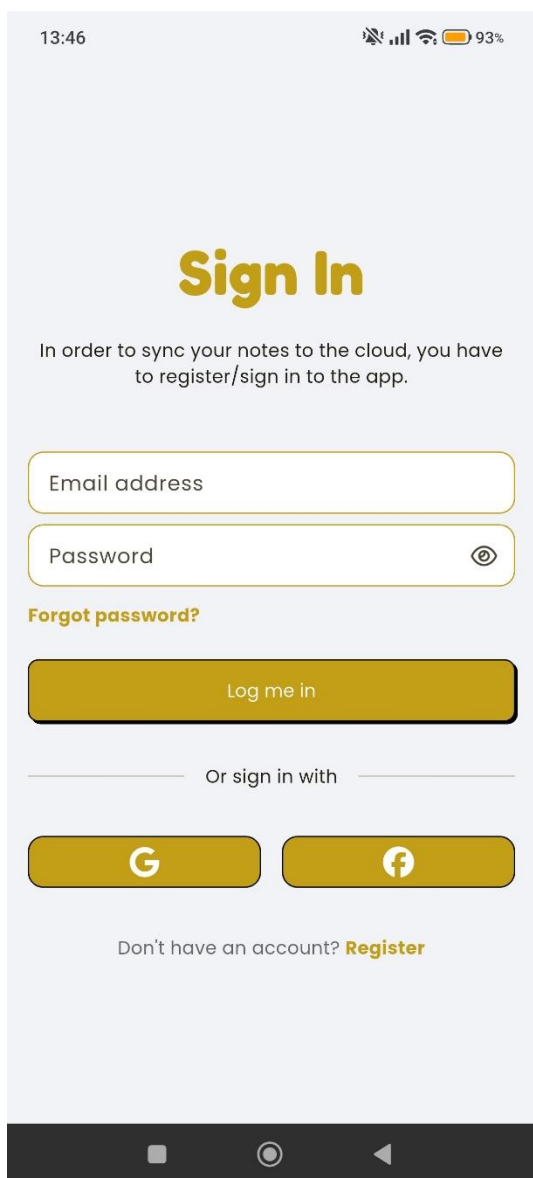
Тази страница предоставя възможност на нерегистрирания потребител да създаде профил в системата. За успешно регистриране е необходимо да попълни задължителните полета – „Пълно име“, „Имейл адрес“ и „Парола“. Има опция за регистриране чрез Google и Facebook също. След като данните бъдат въведени коректно, потребителят може да влезе в системата през страницата за вход („Sign In“). На фигура 9 е представен примерен изглед на страницата за регистрация.



Фигура 9. Страница за регистрация на апликацията „Flutter Notes“

3.2.2. Страница „Sign In“

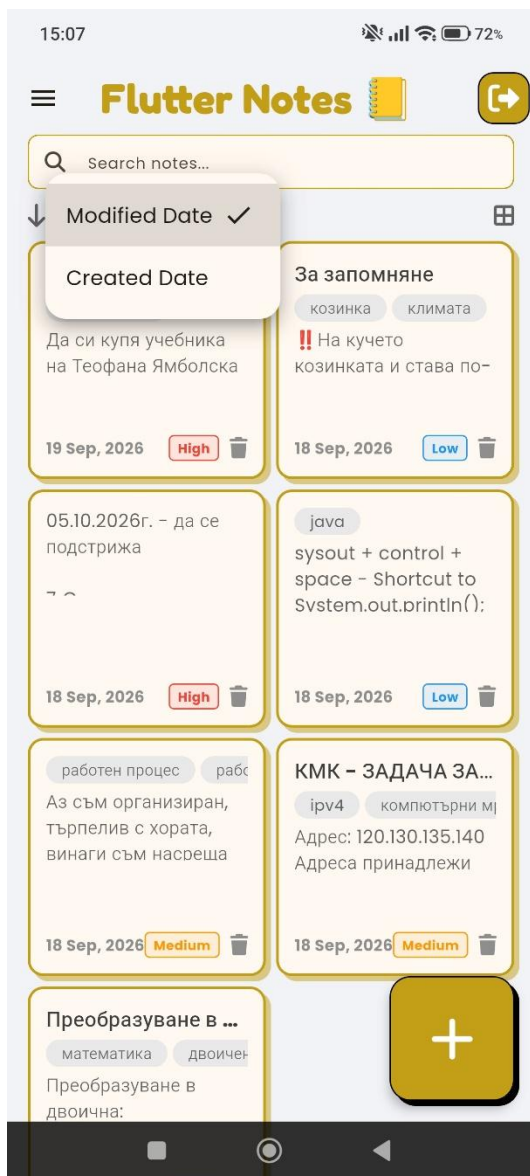
Тази страница предоставя възможност на потребителя да влезе в системата чрез имейл адрес и парола. Предвидена е функционалност за запазване на сесията, която осигурява автоматично разпознаване на потребителя при последващи посещения. При забравена парола е наличен бутонът „Забравена парола?“, чрез който потребителят може да зададе нова такава за профила си. Ако въведените имейл и парола не съответстват на съществуващ потребител, се извежда съобщение за грешка. На фигура 10 е представен примерен изглед на страницата за вход, която позволява на потребителя да осъществи достъп до системата.



Фигура 10. Страница за вписване в приложението на „Flutter Notes“

3.2.3. Филтриране на бележките по дата на промяна

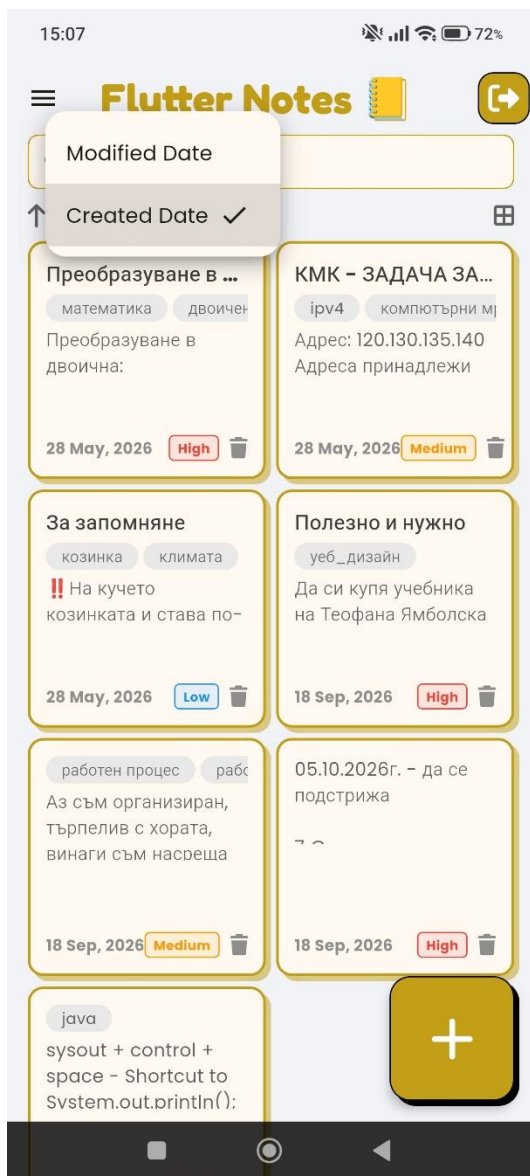
Тази функционалност е разработена, за да може да се филтрират бележките по дата на промяна или по дата на създаване (по-надолу изобразено), за да сме в „крак“ с това какво се случва с всяка бележка. Като филтъра разполага с низходяща и възходяща подредба. На фигура 11 е изобразен примерния изглед на филтриране по дата на промяна в низходяща подредба.



Фигура 11. Филтриране по дата на промяна на бележката

3.2.4. Филтриране на бележките по дата на създаване

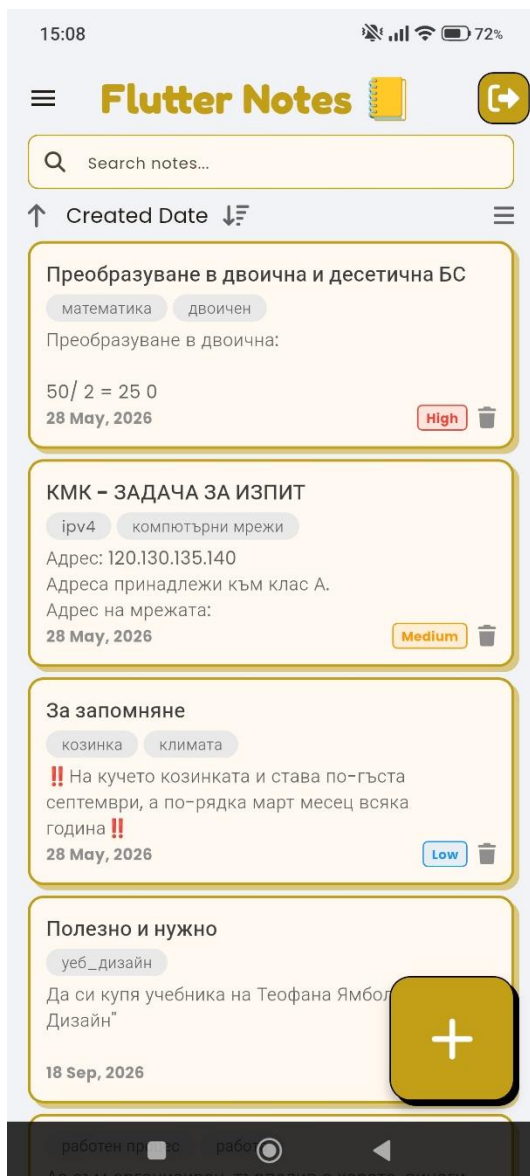
На фигура 12 е изобразен примерния изглед на филтриране по дата на създаване на бележка във възходяща подредба.



Фигура 12. Филтриране по дата на създаване на бележката

3.2.5. Промяна на изгледа на бележките

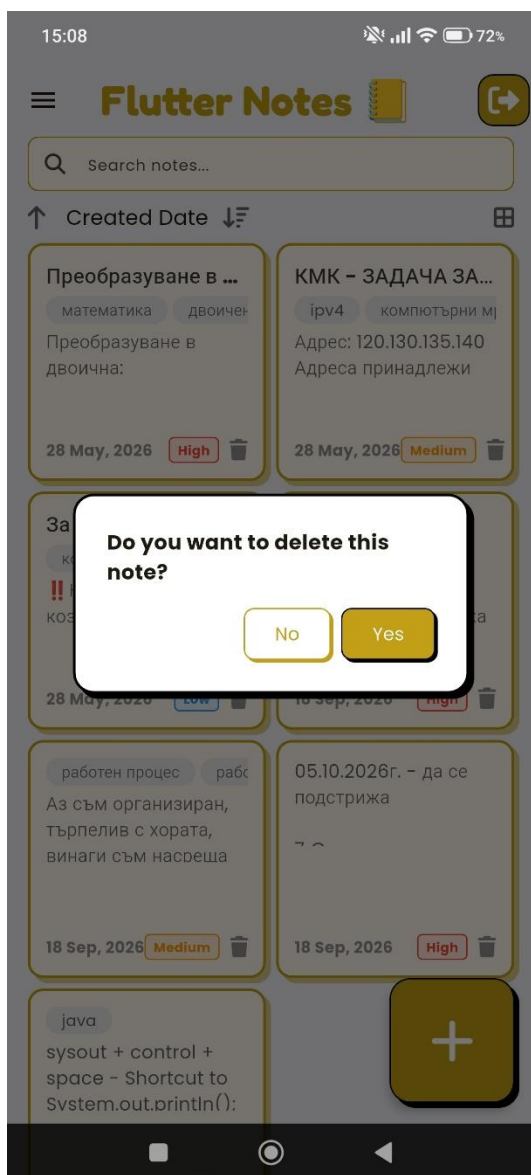
Тази функционалност е разработена, за да може да се променя изгледа на бележките, защото има различни типове хора, които например искат изгледа на бележките да им е различен и искат да си го променят за разнообразие. Трябва да се има и на предвид, че има и хора, които не могат да виждат добре и да се ориентират, така че тази функционалност е подходяща за тях. На фигура 13 е изобразен примерния изглед на промяна на бележките.



Фигура 13. *Промяна на изгледа на бележките*

3.2.6. Изтриване на бележка

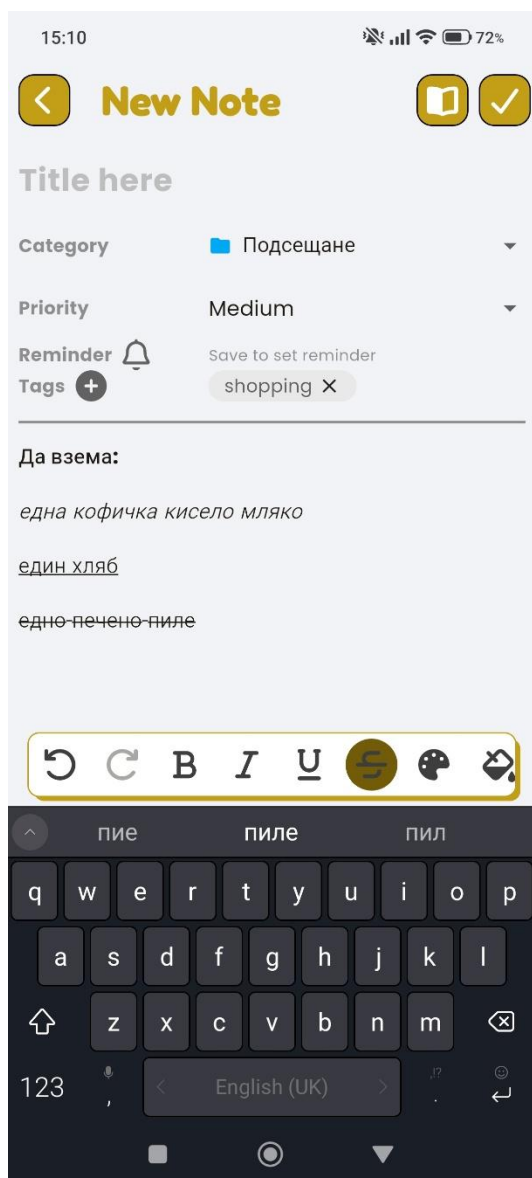
Разбира се бележките могат да се изтриват, когато вече станат ненужни или неизползваеми. На фигура 14 е показан прозореца, който излиза, когато искаш да изтриеш дадена бележка и натиснеш преди това бутона с кошче в долния десен ъгъл на бележката.



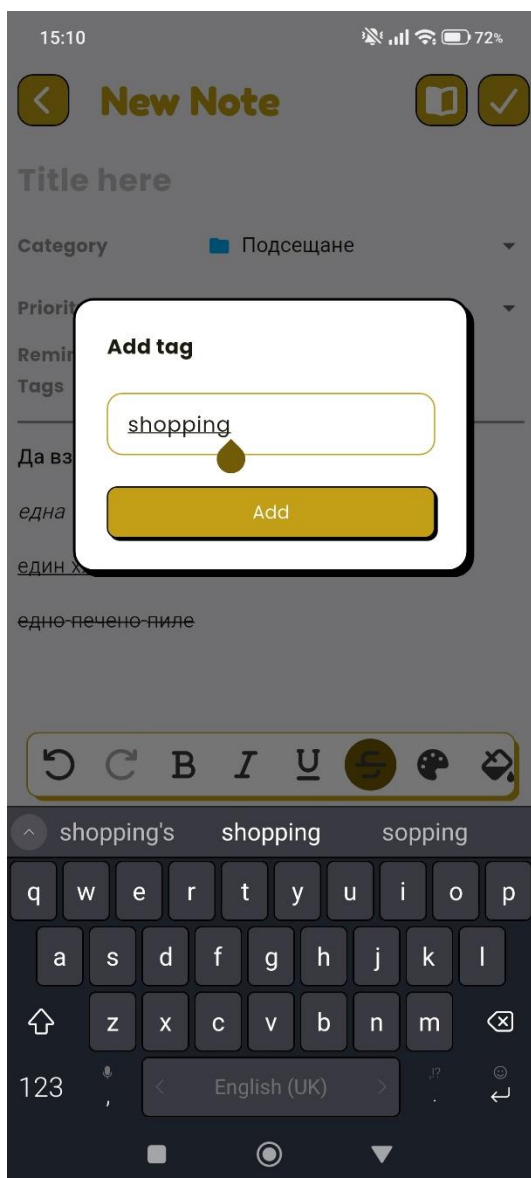
Фигура 14. Изтриване на бележка

3.2.7. Добавяне на бележка

Когато натиснеш бутона за добавяне на бележка потребителя бива пренасочван към екрана за създаване на такава. Там задължително трябва да се попълни Заглавие на бележката, след това може да си създадете тагове за ориентиране на записките. Чрез тях ще се подсеща по-лесно крайния потребител коя бележка му трябва да избере. Най-накрая потребителя може да въвежда неограничен набор от текст в съдържанието на бележката и да го форматира и стилизира както иска. На фигура 15 е показан примерен изглед на бележка, която бива добавяна постепенно. На фигура 16 е показано как се добавя ориентиrowъчен таг към бележката.



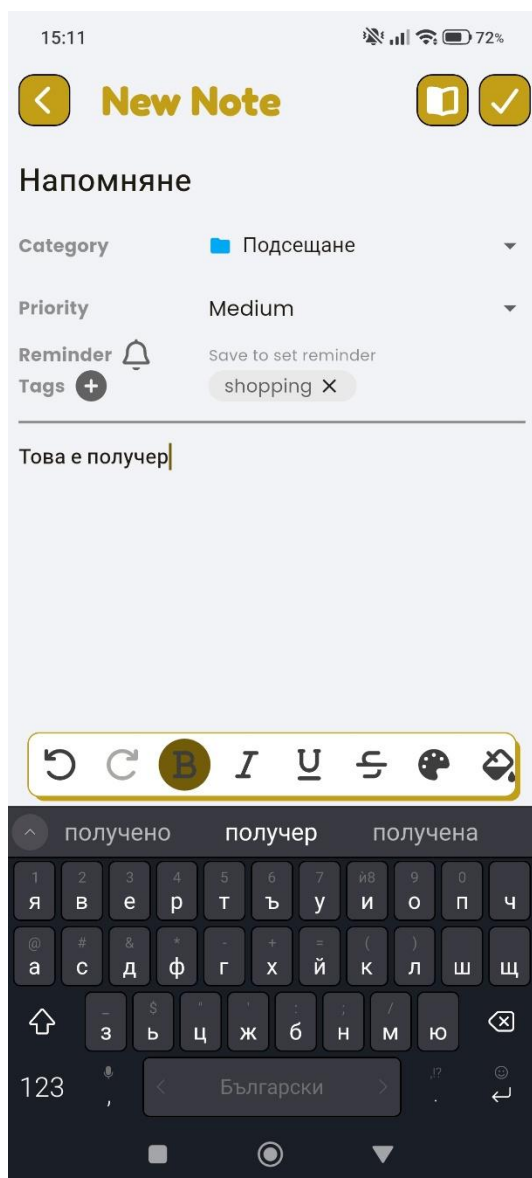
Фигура 15. Добавяне на бележка



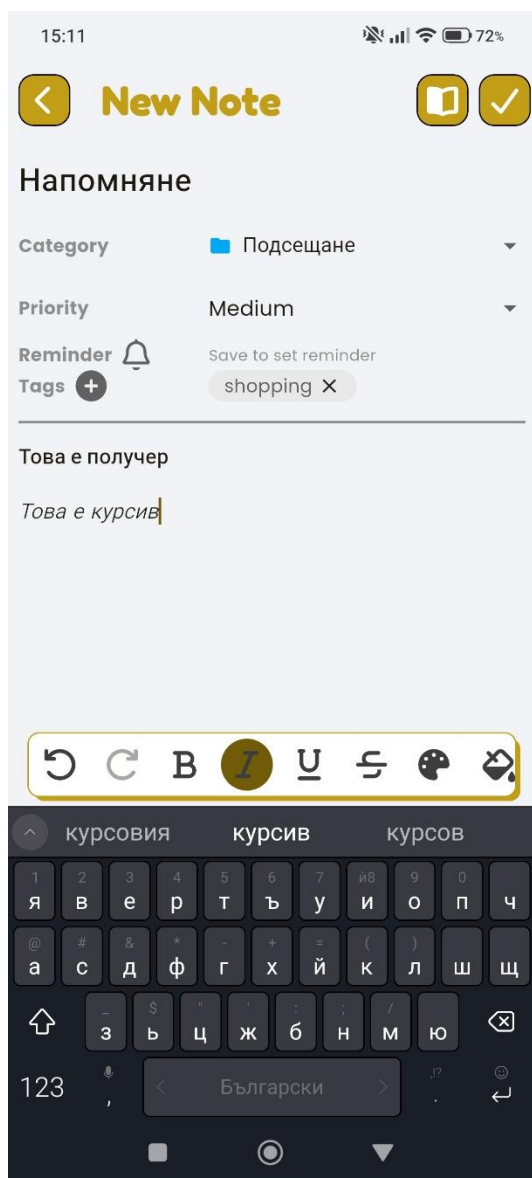
Фигура 16. *Добавяне на таг към бележка*

3.3. Функционалности за форматиране и стилизиране на бележките

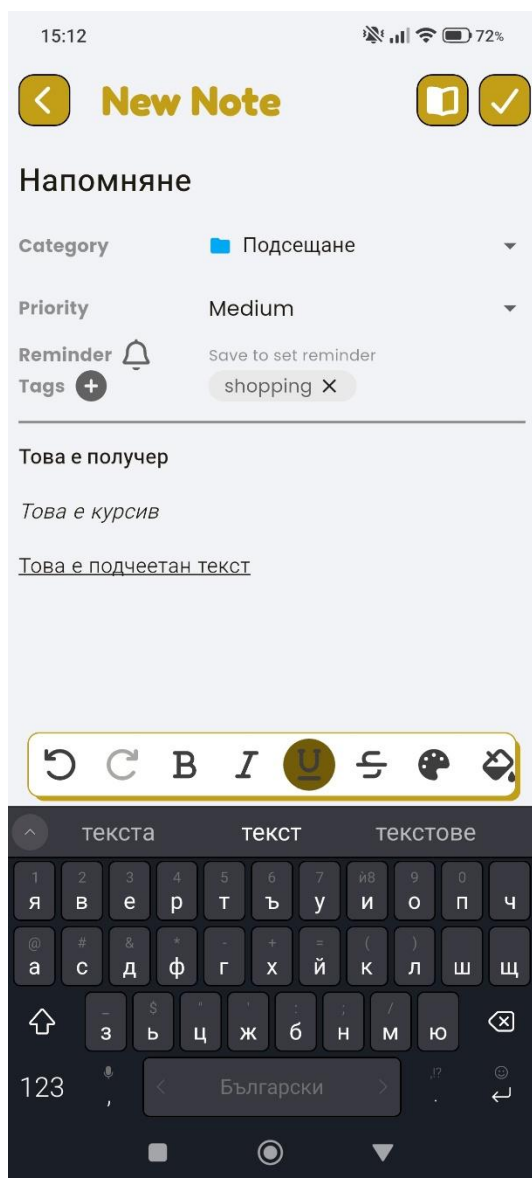
Сред функционалностите за форматиране и стилизиране на бележки присъстват така познатите ни получер (Фигура 17), курсив (Фигура 18), подчертан текст (Фигура 19), зачертан текст (Фигура 20), смяна на цвета на текста (Фигура 21), смяна на фона на текста (Фигура 22), създаване на списъци - тип арабски цифри (Фигура 23), тип кръг (Фигура 24). По-нататък е изобразена цялостната бележка с всички функционалности за стилизиране с възможност за редактиране (Фигура 25).



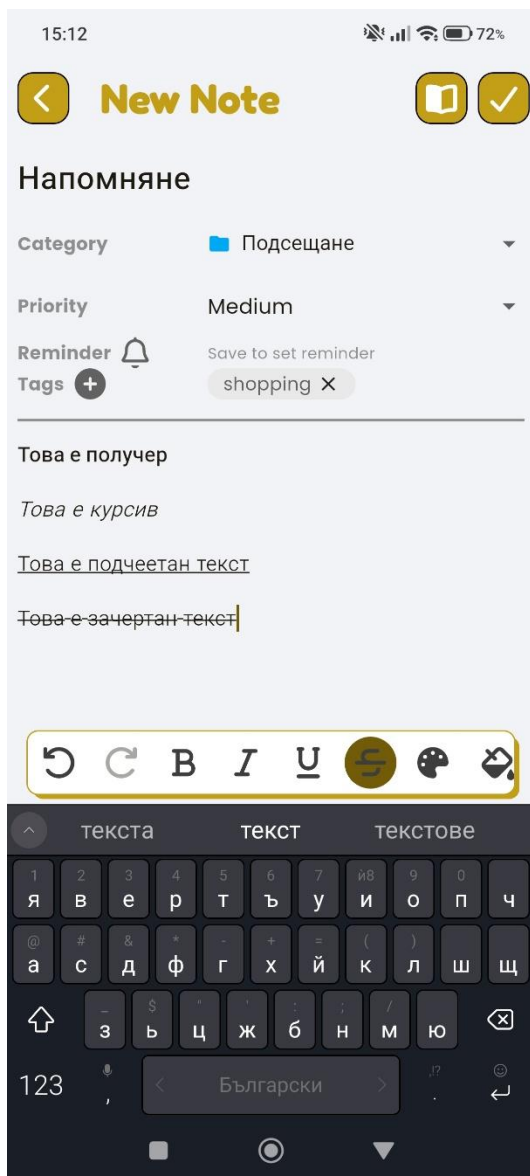
Фигура 17. Функция „получер“



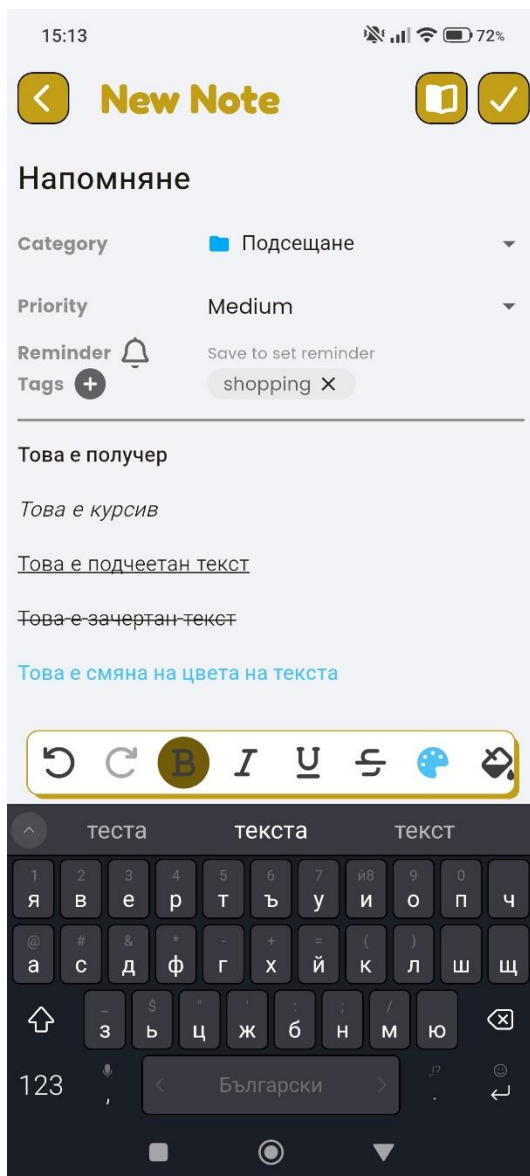
Фигура 18. Функция „курсив“



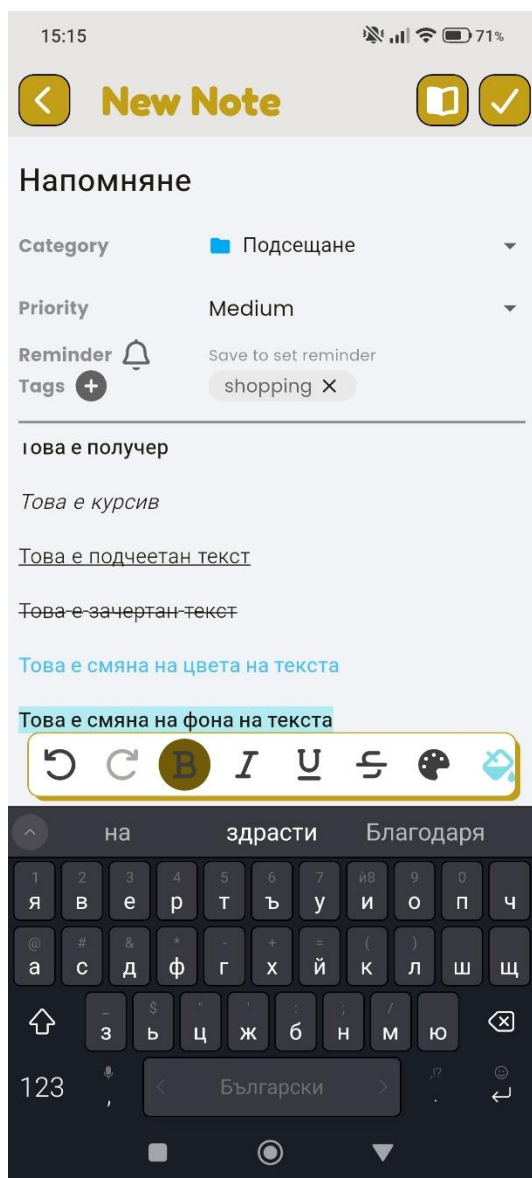
Фигура 19. Функция „подчертан текст“



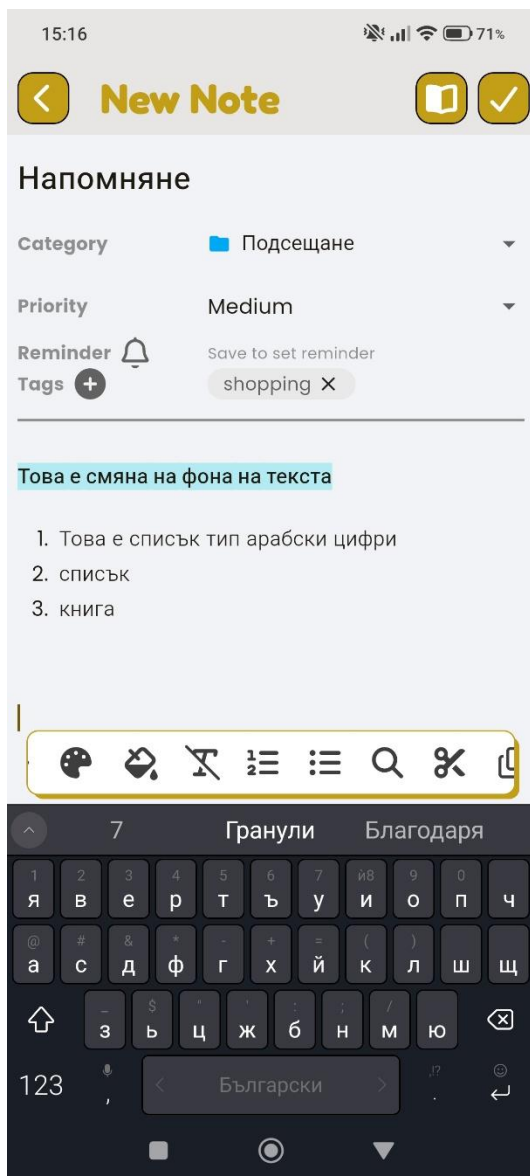
Фигура 20. Функция „зачертан текст“



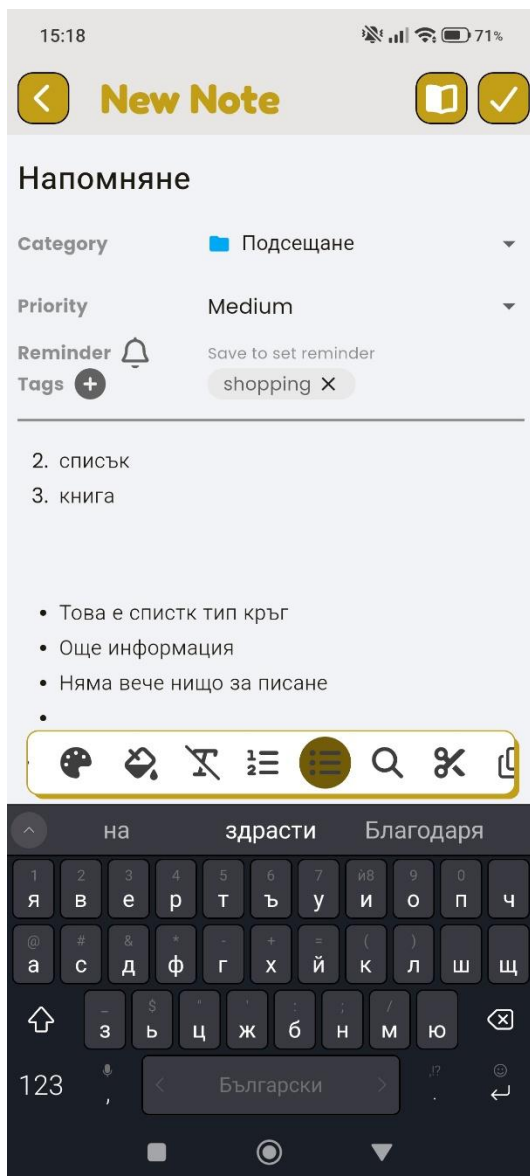
Фигура 21. Функция за смяна на цвета на текста



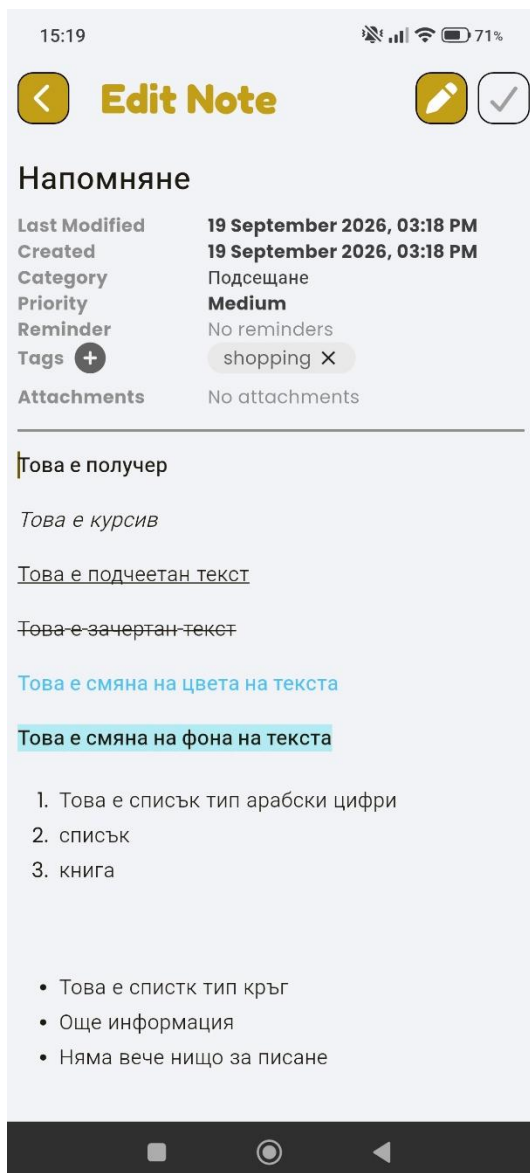
Фигура 22. Функция за смяна на фона на текста



Фигура 23. Създаване на списък тип „Арабски цифри“

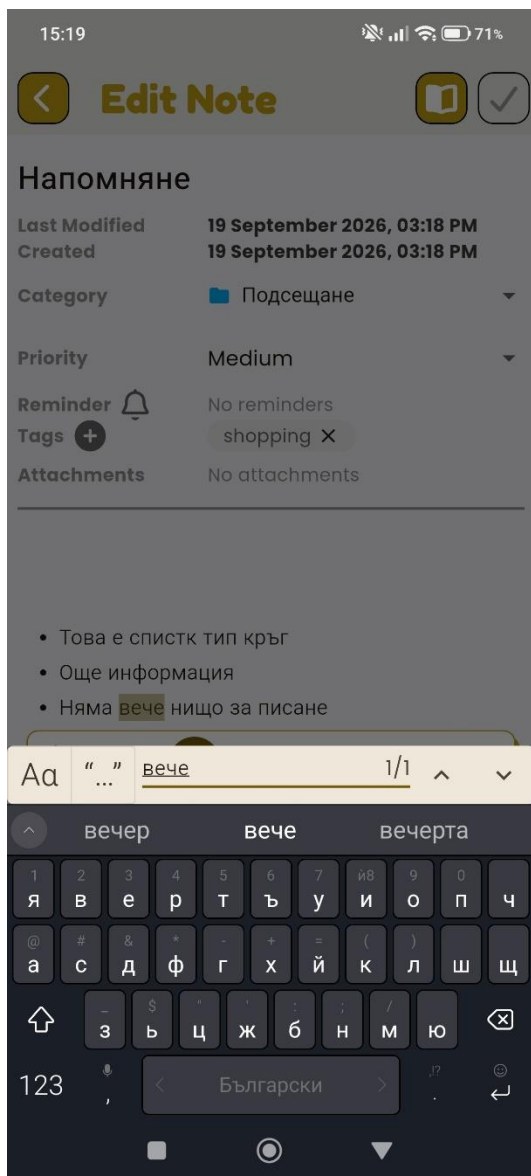


Фигура 24. Създаване на списък тип „Кръг“



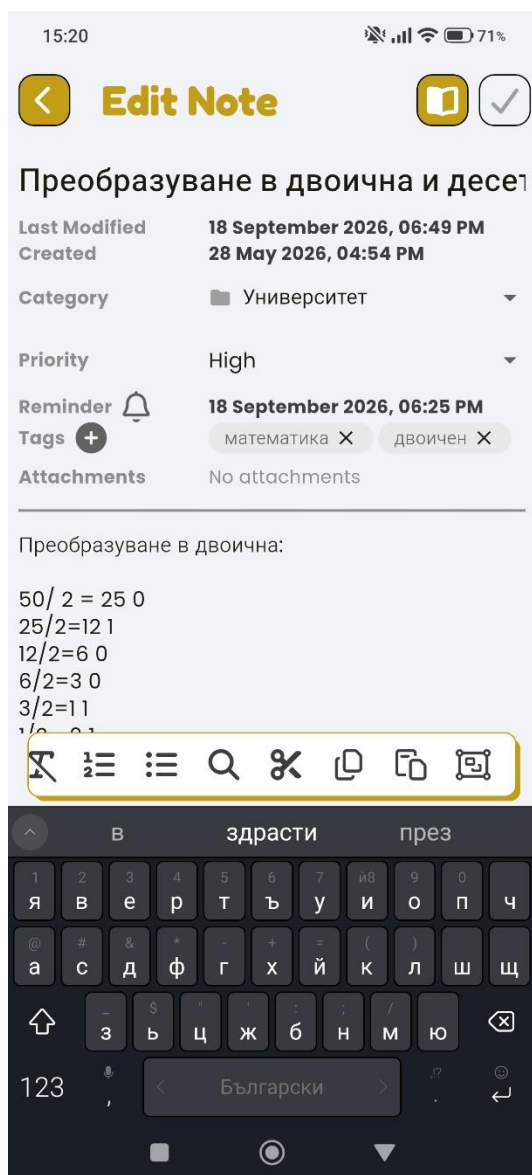
Фигура 25. Краен резултат с възможност за редактиране на бележката

Менюто от функционалности разполага и също със търсачка за търсене на съвпадения в текста, понеже понякога имаме нужда да потърсим нещо, което сме видели и искаме по-лесно да се ориентираме (Фигура 26).



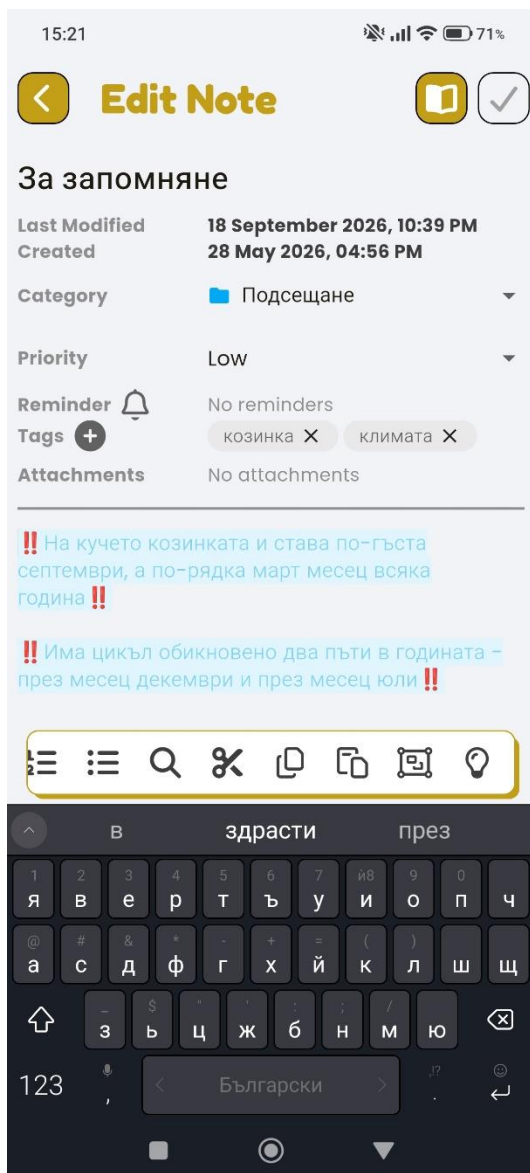
Фигура 26. *Търсачка за съвпадения в текста на бележката*

Бяха реализирани също функционалности за Cut, Copy, Paste и Select All, за да е по-удобно на потребителя да си съхранява бележките и да си ги прехвърля от една на друга платформа. (Фигура 27)

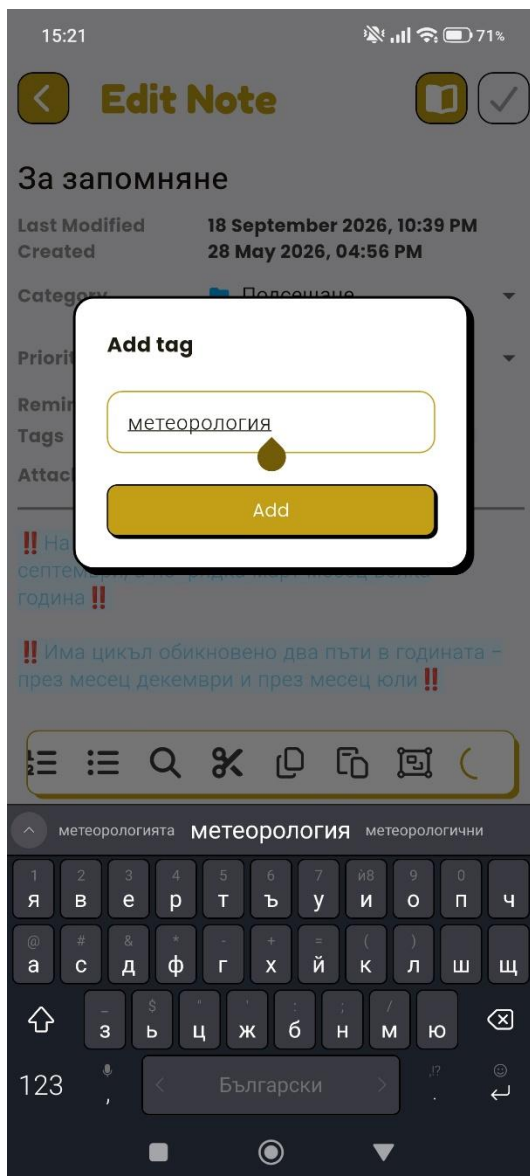


Фигура 27. Функционалностите за Cut, Copy, Paste и Select All

Беше реализирана също и функционалност за интелигентно предлагане на тагове на всяка бележка индивидуално, като беше интегриран приложен програмен интерфейс (API) на големия езиков модел (LLM) Llama на компанията Meta, понеже самия интерфейс беше безплатен за ползване според плана на документацията на Cloudflare, която ползвах (Cloudflare Workers AI). В зависимост от това какво има написано във всяка бележка индивидуално се отваря нов прозорец за добавяне на таг, когато натиснем бутона за функцията tag suggestion и Llama ни предлага нов таг. (Фигура 28 и 29)

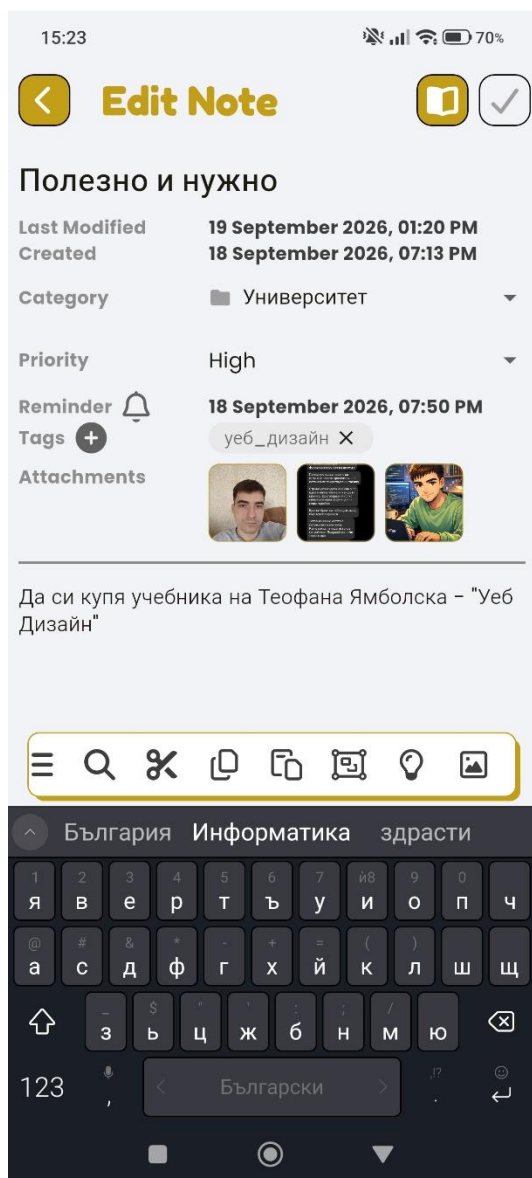


Фигура 28. Функционалността за *tag suggestion* с Изкуствен интелект

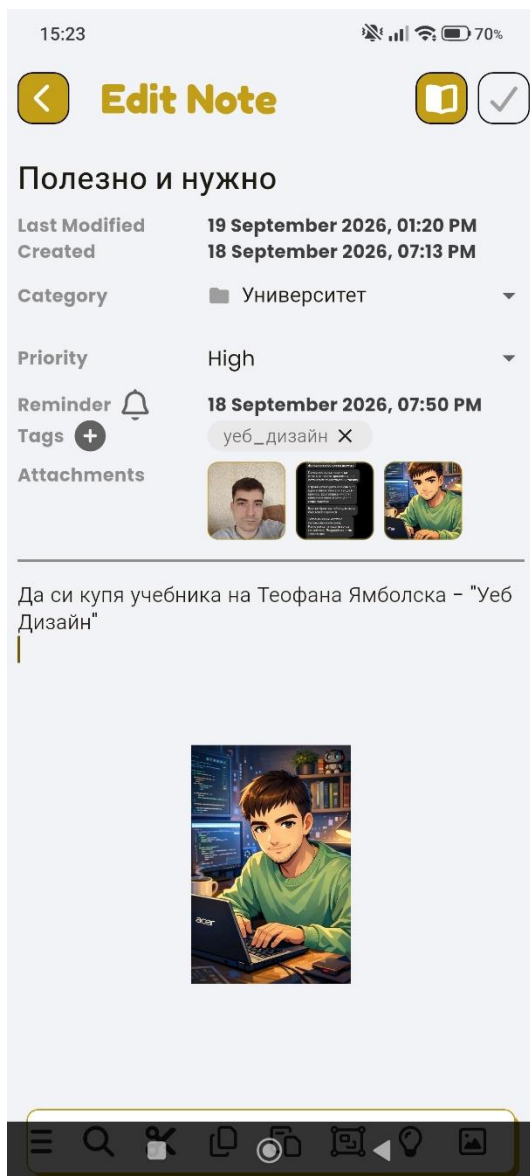


Фигура 29. *Как работи функцията за tag suggestion*

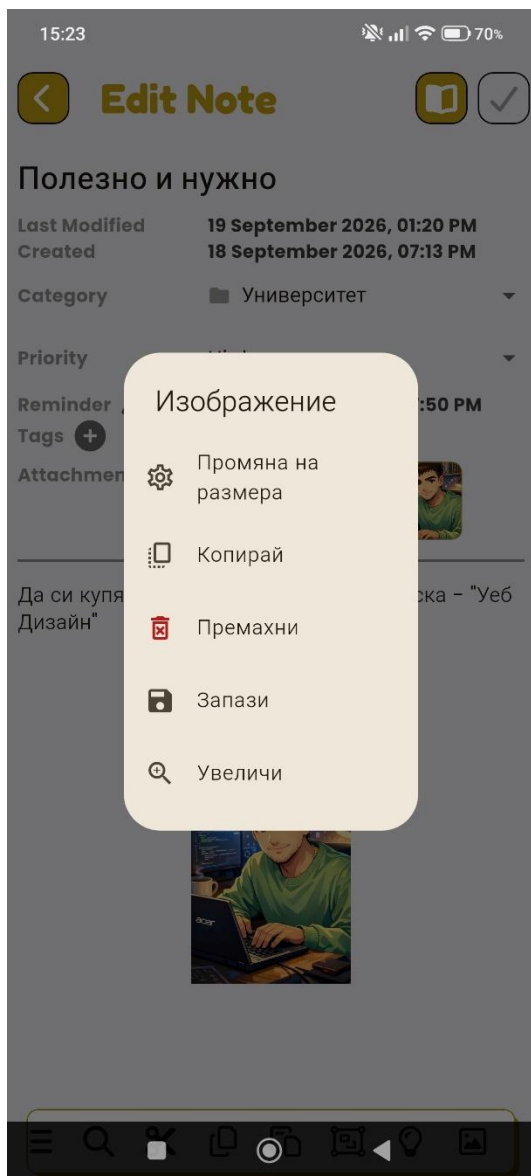
Беше реализирана също функционалност за прикачване на файлове към дадена бележка с функция "Добави снимка" в тулбара на бележката. Файлът се качва в Firebase Storage и е видим във бележката, която си го поставил, като разбира се може да се променя размера, да се копира, да се премахва, да се запазва и да се увеличава във пълен размер, а линкът се записва тук, свързан с `noteld`. Функционалността е показана на фигури 30, 31, 32, като е визуализиран накрая диалоговия прозорец при натискане на изображението.



Фигура 30. Функцията за добавяне на изображение в бележка



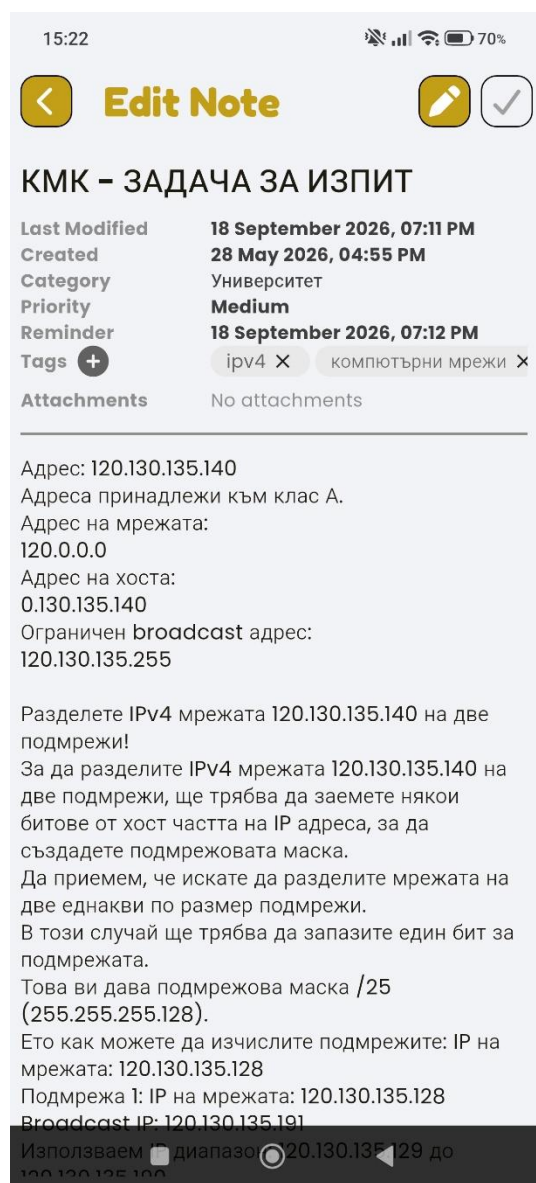
Фигура 31. Добавено изображение, което е и в мален размер



Фигура 32. Диалоговия прозорец при натискане на изображението

Не на последно място беше и реализирана и функционалност за напомняния(reminders). Потребителят избира дата и час за известяване към конкретна бележка. При добавяне, приложението планира локално или Push известие. Също така и към всяка бележка може да се добавя приоритет. Приоритетите биват следните: Low, Medium, High.

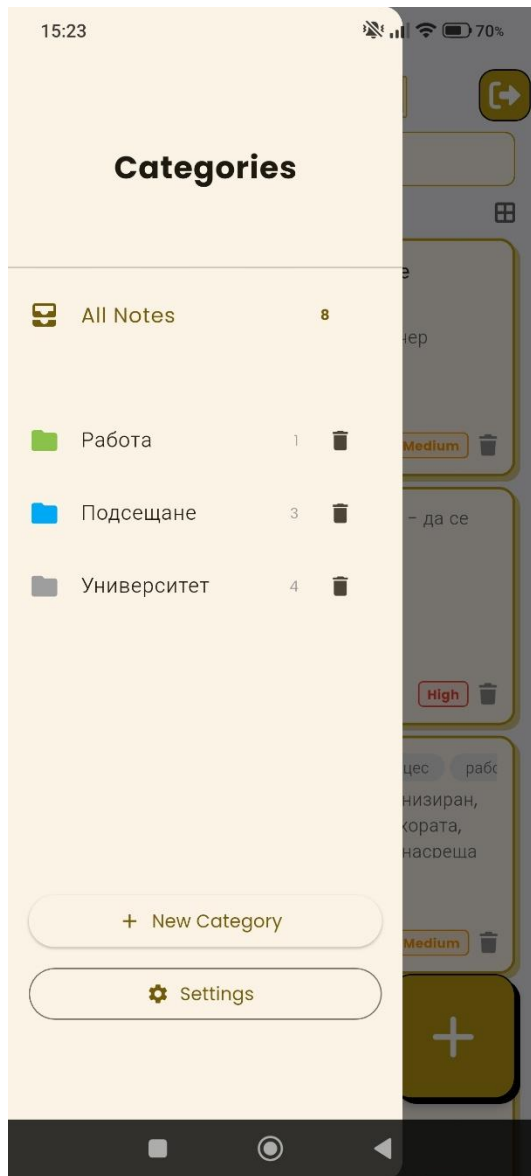
Тук ще предоставя завършена бележка за онагледеност как би изглеждала (Фигура 30).



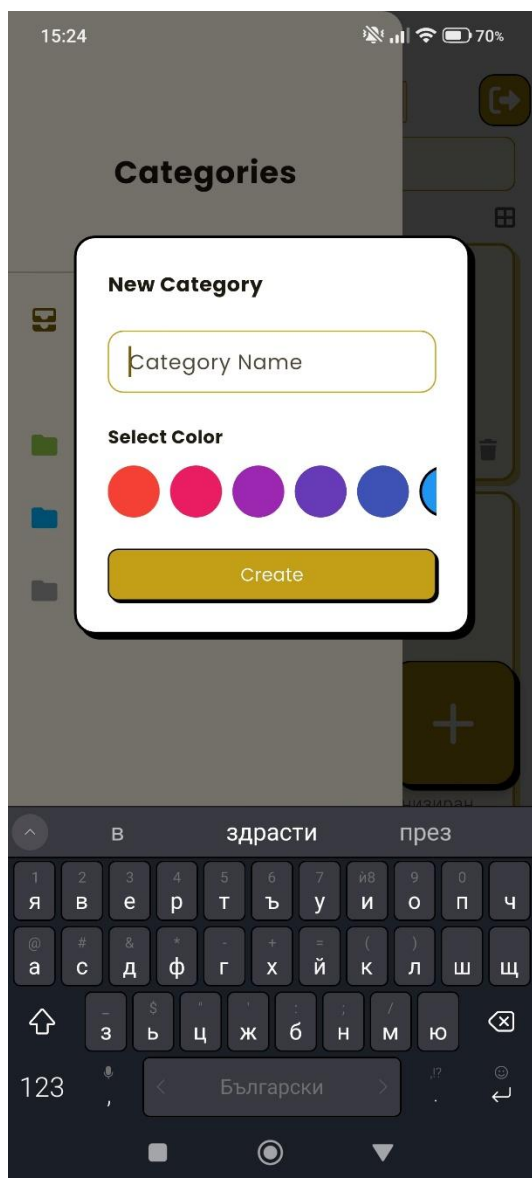
Фигура 33. Примерна бележка

Беше реализиран и таб, в който се достъпват два бутона. Единият бутон е за добавяне на категория или т.н. Папка, в която се съхраняват бележки, а другият е за достъпване на Потребителските настройки, от които се сменя режима, като той може да е Тъмен или Светъл режим. На фигура 34 е показан таба, на фигура 35 е показан резултата, след като се натисне бутона New Category, на фигура 36

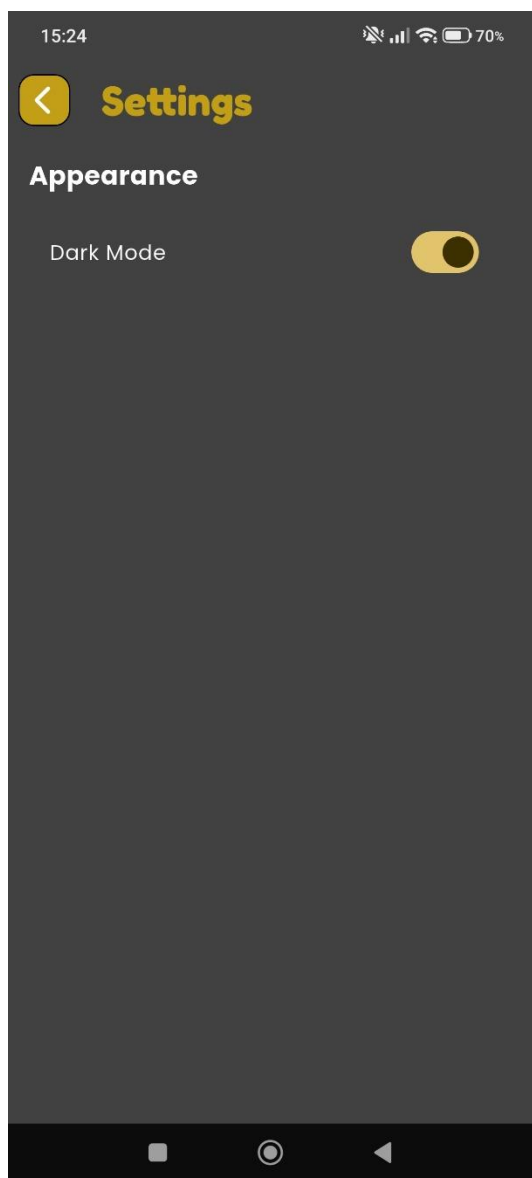
и 37 е показан раздела Потребителски настройки и изгледа на началната страница при включен Тъмен режим.



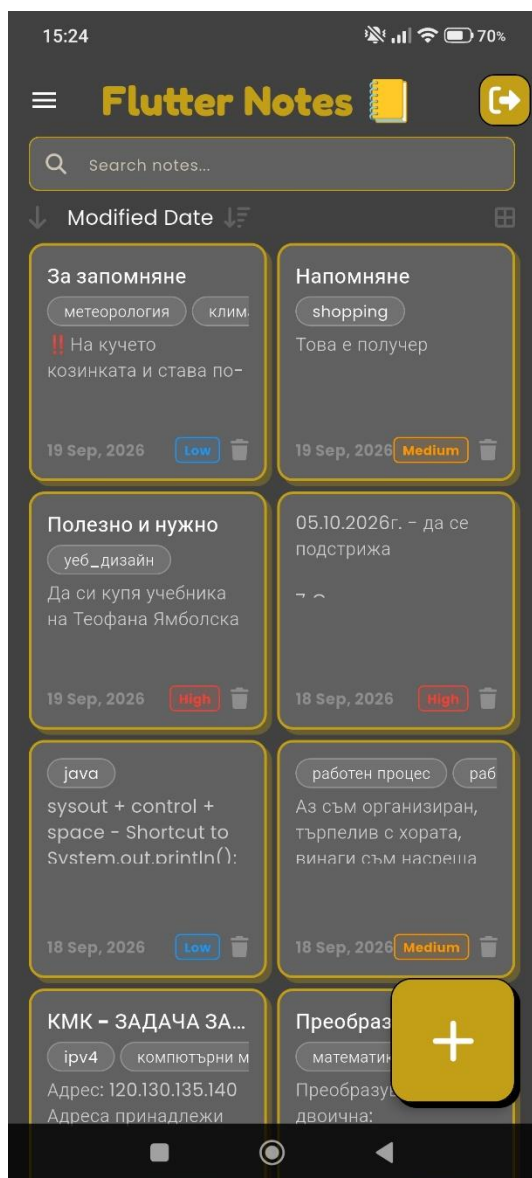
Фигура 34. Това е добавения нов таб



Фигура 35. Това е резултата от натискането на бутона “New Category”



Фигура 36. Тук е показан раздела „Потребителски настройки“



Фигура 37. Това е изгледа на началната страница при включен Тъмен режим

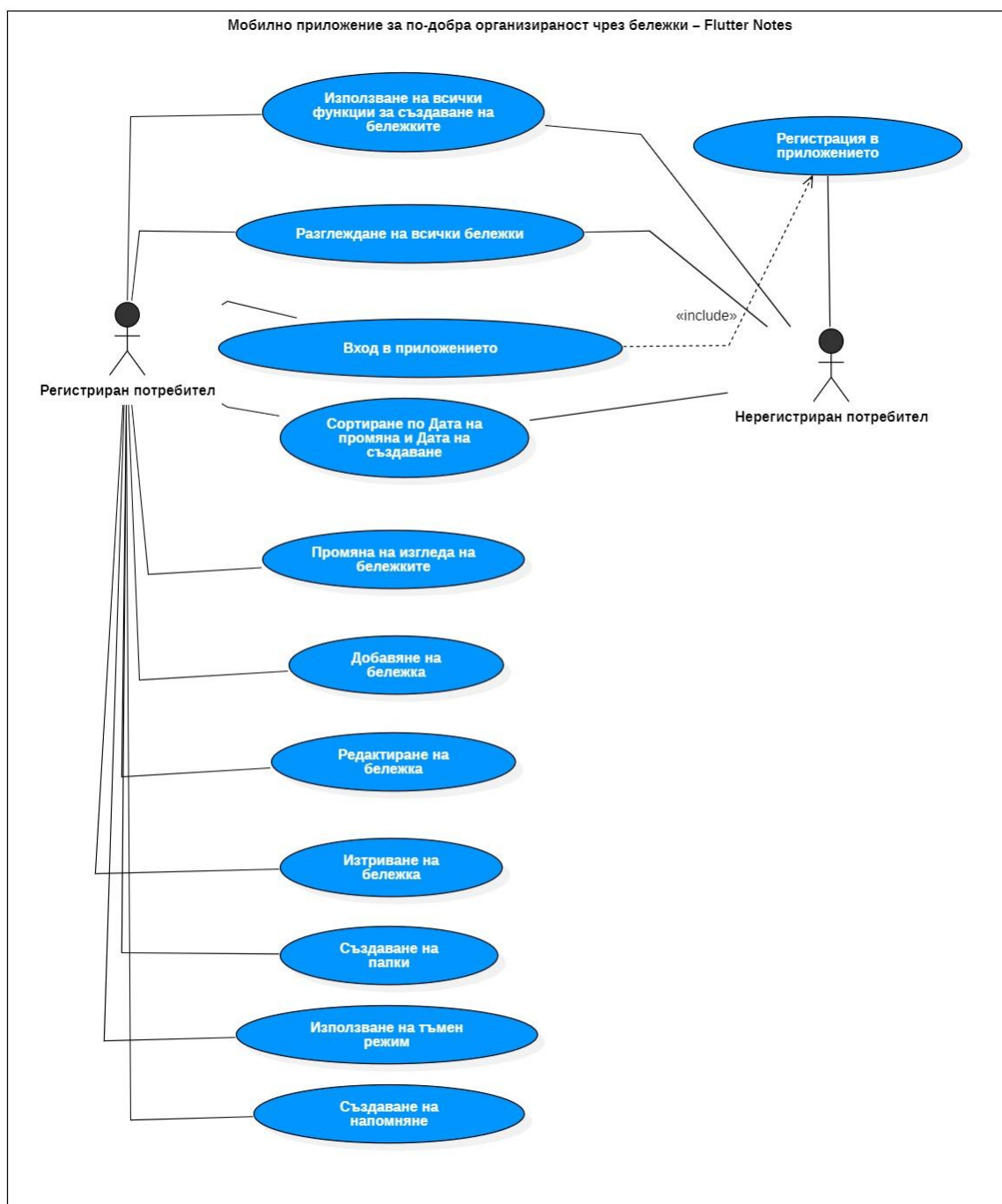
3.4. UML Диаграми на приложението

Унифицираният език за моделиране (UML) е графичен език, използван за визуализиране, специфициране, конструиране и документиране на елементите в софтуерната система. UML включва 13 официални типа диаграми, като елементи от един тип могат да се използват и в друг. Те са следните:

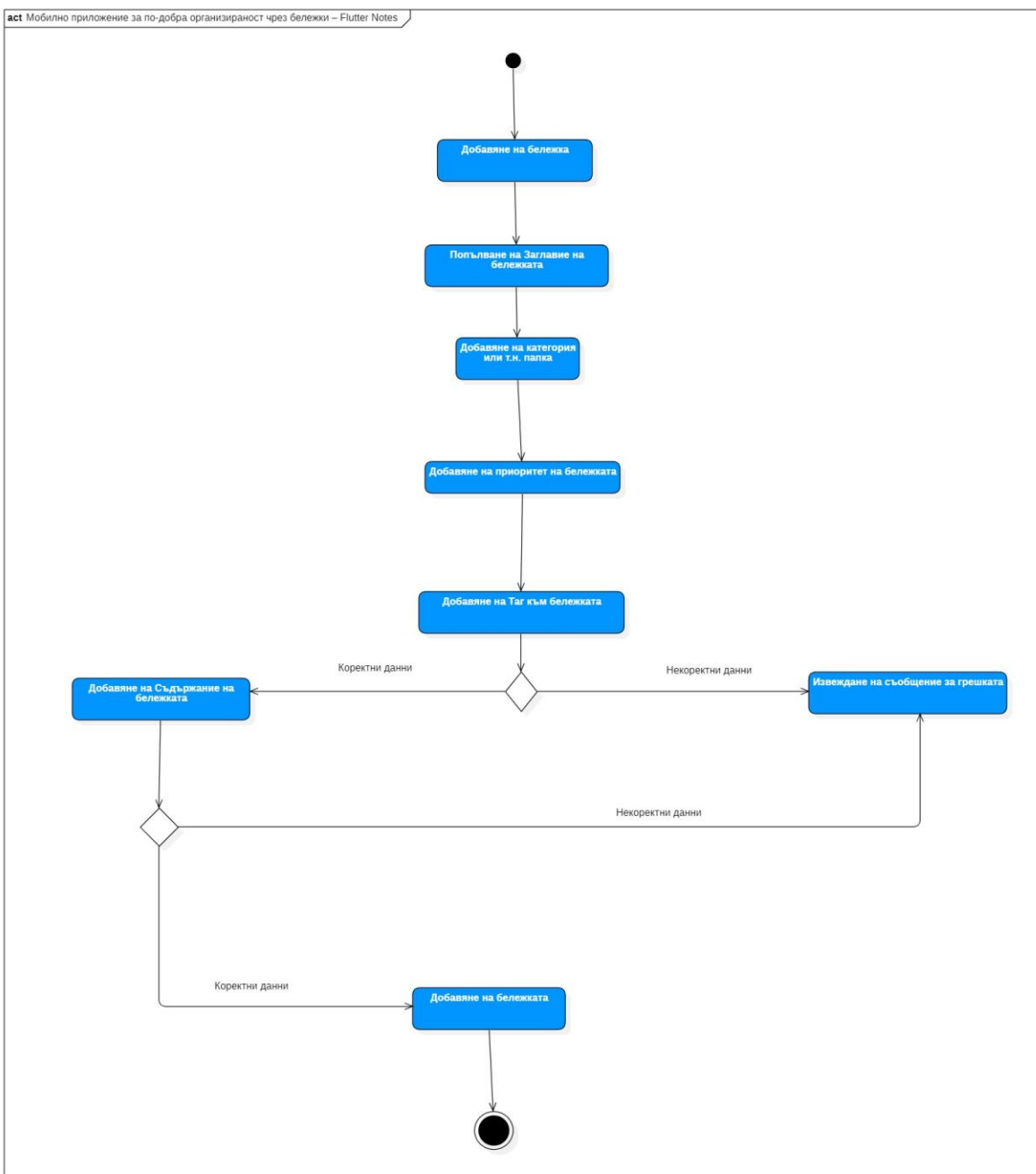
- Диаграма на дейностите (Activity Diagram) – описва процедурното и паралелното поведение;
- Диаграма на класовете (Class Diagram) – представя класовете от обекти и връзките между тях;
- Диаграма на комуникацията (Communication Diagram) – описва взаимодействията между обектите с акцент върху връзките;
- Диаграма на компонентите (Component Diagram) – показва структурата и взаимовръзките между компонентите;
- Диаграма на съставни структури (Composite Structure) – използва се за декомпозиция на клас по време на изпълнение;
- Диаграма за разгръщане (Deployment Diagram) – визуализира разположението на артефактите във възлите;
- Диаграма за преглед на взаимодействията (Interaction Overview) – комбинира диаграми на последователност и дейности;
- Диаграма на обектите (Object Diagram) – представя примерни конфигурации на инстанции;
- Диаграма на пакетите (Package Diagram) – показва йерархичната структура по време на компилиране;
- Диаграма на последователността (Sequence Diagram) – илюстрира взаимодействията между обектите с акцент върху времевата последователност;
- Диаграма на състоянията (State Machine Diagram) – описва промените в обекта под въздействието на събития през неговия жизнен цикъл;

- Диаграма за време (Timing Diagram) – представя взаимодействията между обектите с фокус върху синхронизацията;
- Случай на употреба (Use Case Diagram) – показва взаимодействието между потребителя и системата.

На фигура 38 е представена UML Use case диаграма, която визуализира целия процес в мобилното приложение Flutter Notes, включително различните потребителски роли и техните права за достъп. На фигура 39 е показана UML Activity диаграма, която описва поведението на обектите при добавяне на ново напомняне и последователността от стъпки, водещи до успешното му публикуване.



Фигура 38. UML Use Case Диаграма, визуализираща цялостният процес в приложението



Фигура 39. UML Activity Диаграма, описваща добавянето на нова бележка

3.5. Описание на дизайна

Преди започване на разработката на мобилното приложение е от съществено значение да се изгради ясна представа за неговия визуален облик и дизайн. Първостепенно значение при избора на дизайн има тематичната насоченост на платформата и нейното предназначение. Разглежданото мобилно приложение за бележки – Flutter Notes, спада към категорията на помощните и организационни инструменти, поради което се набляга на опростен, интуитивен и функционален интерфейс, който улеснява потребителите в ежедневната им работа.

3.5.1. Избор на цветове

Цветовото оформление на едно мобилно приложение е от ключово значение за цялостното потребителско изживяване. Чрез цветовете се изразяват емоции, нагласи и асоциации, които влияят пряко върху възприемането на съдържанието. Психологията на цветовете намира широко приложение в дизайна, тъй като цветовете имат силата да предизвикват различни чувства и реакции у ползвателите. Затова изборът на подходяща цветова гама е сред основните решения при изграждането на всяко приложение.

Цветовете могат да бъдат класифицирани с помощта на цветното колело, което представлява логическо подреждане на чистите цветове и демонстрира взаимовръзките между тях. Съществуват няколко основни типа цветови схеми – монохроматични, аналогови, допълващи се и персонализирани. Монохроматичните схеми се основават на един основен цвят и неговите нюанси, което не натоварва зрително и успокоява потребителите. Аналоговите схеми използват съседни цветове от колелото, подходящи за степенуване на съдържание. Допълващите се схеми разчитат на контрастни цветове за привличане на вниманието, а персонализираните предлагат свобода за творчество, като често съчетават неутрални тонове с един ярък акцент.



Фигура 40. *Цветно колело*

В настоящата разработка специално внимание е отделено на цветовата палитра, която да отговоря на нуждите и очакванията на крайните потребители. Основният цвят, използван в приложението, е жълтото, допълнено от бяло и черно. Жълтият цвят символизира светлина, позитивизъм, енергия и креативност. Той стимулира умствената дейност и създава усещане за оптимизъм и топлина, което го прави особено подходящ за приложение, свързано с организираност и продуктивност.

Бялото асоциира с чистота, простота и яснота, като осигурява визуално пространство и подчертава съдържанието. Черното, от своя страна, придава контраст, елегантност и структурираност, като

спомага за по-добрата четимост на текста и ясното разграничаване на елементите.

Комбинацията на жълто, бяло и черно създава баланс между топлина и строгост, между динамичност и прегледност. Тя подпомага фокусирането върху съдържанието, улеснява навигацията и допринася за приятно и запомнящо се потребителско изживяване. Именно поради тези характеристики, избраната цветова палитра е оптимална за разработваното мобилно приложение за бележки.

Глава четвърта – Реализация и програмен код

4.1. Реализация на базата данни

Базата данни, която е реализирана в приложението „Flutter Notes“, съдържа 8 основни колекции.

4.1.1. Колекция Потребители (users)

Съдържа всички характеристики, които един регистриран потребител притежава – уникален идентификатор на потребител, пълно име, имейл адрес и парола.

4.1.2. Колекция Бележки (notes)

Съдържа всички характеристики, които всяка бележка притежава – уникален идентификатор на бележка, уникален идентификатор на потребител, уникален идентификатор на категория, заглавие, съдържание, съдържание в формат JSON, приоритет на бележката, дата на създаване, дата на промяна.

4.1.3. Колекция Категории (categories)

Съдържа всички характеристики, които всяка папка притежава – уникален идентификатор на папка, уникален идентификатор на потребител, име, име на иконата, цвят в HEX код.

4.1.4. Колекция Тагове (tags)

Съдържа всички характеристики, които всеки таг в дадена бележка притежава – уникален идентификатор на таг, уникален идентификатор на потребител, етикет.

4.1.5. Колекция връзка Бележка – Таг (note_tags)

Съдържа връзките между бележките и таговете, като служи за реализиране на връзка „много към много“ между тях. Таблицата се състои от съставен първичен ключ, образуван от уникален идентификатор на бележка и уникален идентификатор на таг.

4.1.6. Колекция Потребителски настройки (app_preferences)

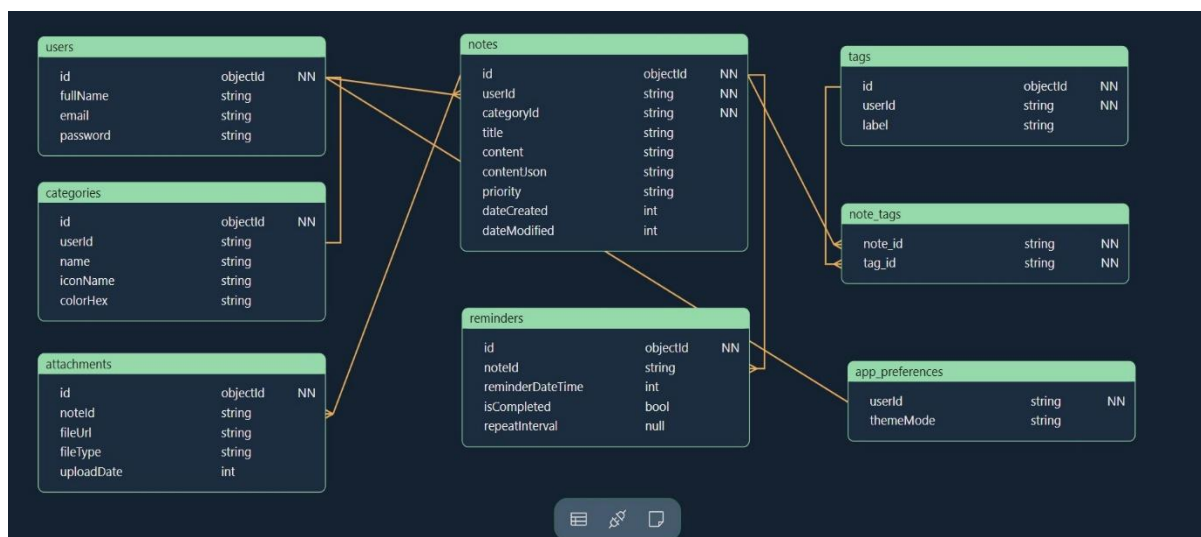
Съдържа персоналните настройки на всеки потребител – уникален идентификатор на потребител (който е и първичен ключ) и избран режим на темата (светла/тъмна).

4.1.7. Колекция Напомняния (reminders)

Съдържа информация за напомнянията, свързани с дадена бележка – уникален идентификатор, идентификатор на бележка, дата и час на напомняне, статус на изпълнение и интервал на повторение.

4.1.8. Колекция Прикачени файлове (attachments)

Съдържа метаданни за файловете, прикачени към бележките – уникален идентификатор, идентификатор на бележка, път до файла (URL), тип на файла и дата на качване.



Фигура 41. Схема на базата данни на приложението

4.2. Програмна реализация

В тази секция е представен програмния код с обяснения на някои от основните градивни компоненти на мобилното приложение. Както подробно е описано в глава втора, подточка 2.1, приложението е разработено и написано на програмния език Flutter(Dart).

4.2.1. Код за сървиз на бележка

```
import 'package:cloud_firestore/cloud_firestore.dart';
import 'package:flutter/foundation.dart';
import '../models/note.dart';
import 'auth_service.dart';

class NoteService {
  static final FirebaseFirestore _firestore = FirebaseFirestore.instance;

  // Референция към колекцията на текущия потребител
  static CollectionReference get _userNotesCollection {
    final userId = AuthService.user?.uid;
    if (userId == null) {
      throw Exception('User not logged in');
    }
    return _firestore.collection('users').doc(userId).collection('notes');
  }

  // Взема всички бележки на потребителя
  static Stream<List<Note>> getNotesStream() {
    return _userNotesCollection
      .orderBy('dateModified', descending: true)
      .snapshots()
      .map((snapshot) {
    return snapshot.docs.map((doc) {
      final data = doc.data() as Map<String, dynamic>;
      return Note(
        title: data['title'],
        content: data['content'],
        contentJson: data['contentJson'],
        dateCreated: data['dateCreated'],
        dateModified: data['dateModified'],
        tags: List<String>.from(data['tags'] ?? []),
      );
    }).toList();
  });
}

// Добавя нова бележка
static Future<void> addNote(Note note) async {
```

```

    await _userNotesCollection.add({
        'title': note.title,
        'content': note.content,
        'contentJson': note.contentJson,
        'dateCreated': note.dateCreated,
        'dateModified': note.dateModified,
        'tags': note.tags ?? [],
    });
}

// Актуализира съществуваща бележка
static Future<void> updateNote(Note note) async {
    // Намираме документа по dateCreated (който е уникален)
    final querySnapshot = await _userNotesCollection
        .where('dateCreated', isEqualTo: note.dateCreated)
        .limit(1)
        .get();

    if (querySnapshot.docs.isNotEmpty) {
        await querySnapshot.docs.first.reference.update({
            'title': note.title,
            'content': note.content,
            'contentJson': note.contentJson,
            'dateModified': note.dateModified,
            'tags': note.tags ?? [],
        });
    } else {
        // Ако не се намери, добавяме като нова
        await addNote(note);
    }
}

// Изтрива бележка
static Future<void> deleteNote(Note note) async {
    final querySnapshot = await _userNotesCollection
        .where('dateCreated', isEqualTo: note.dateCreated)
        .limit(1)
        .get();

    if (querySnapshot.docs.isNotEmpty) {
        await querySnapshot.docs.first.reference.delete();
    }
}
}

```

4.2.2. Код за страница за нова бележка или за редактиране на такава

```

import 'package:flutter/material.dart';
import 'package:flutter_quill/flutter_quill.dart';
import 'package:font_awesome_flutter/font_awesome_flutter.dart';
import 'package:provider/provider.dart';

import '../change_notifiers/new_note_controller.dart';
import '../core/constants.dart';
import '../core/dialogs.dart';
import '../widgets/note_back_button.dart';
import '../widgets/note_icon_button_outlined.dart';
import '../widgets/note_metadata.dart';
import '../widgets/note_toolbar.dart';

class NewOrEditNotePage extends StatefulWidget {
  const NewOrEditNotePage({
    required this.isNewNote,
    super.key,
  });

  final bool isNewNote;

  @override
  State<NewOrEditNotePage> createState() => _NewOrEditNotePageState();
}

class _NewOrEditNotePageState extends State<NewOrEditNotePage> {
  late final NewNoteController newNoteController;
  late final TextEditingController titleController;
  late final QuillController quillController;

  late final FocusNode focusNode;

  @override
  void initState() {
    super.initState();

    newNoteController = context.read<NewNoteController>();

    titleController = TextEditingController(text: newNoteController.title);

    quillController = QuillController.basic()
      ..addListener(() {
        newNoteController.content = quillController.document;
      });

    focusNode = FocusNode();
  }

```

```

WidgetsBinding.instance.addPostFrameCallback((timeStamp) {
  if (widget.isNewNote) {
    focusNode.requestFocus();
    newNoteController.readOnly = false;
  } else {
    newNoteController.readOnly = true;
    quillController.document = newNoteController.content;
  }
});
}

```

```

@override
void dispose() {
  titleController.dispose();
  quillController.dispose();
  focusNode.dispose();
  super.dispose();
}

```

```

@override
Widget build(BuildContext context) {
  return PopScope(
    canPop: false,
    onPopInvoked: (didPop) async {
      if (didPop) return;

      if (!newNoteController.canSaveNote) {
        Navigator.pop(context);
        return;
      }

      final bool? shouldSave = await showConfirmationDialog(
        context: context,
        title: 'Do you want to save the note?',
      );

      if (shouldSave == null) return;

      if (!context.mounted) return;

      if (shouldSave) {
        newNoteController.saveNote(context);
      }

      Navigator.pop(context);
    },
    child: Scaffold(
      appBar: AppBar(

```

```

leading: const NoteBackButton(),
title: Text(widget.isNewNote ? 'New Note' : 'Edit Note'),
actions: [
  Selector<NewNoteController, bool>(
    selector: (context, newNoteController) =>
      newNoteController.readOnly,
    builder: (context, readOnly, child) => NoteIconButtonOutlined(
      icon:
        readOnly ? FontAwesomeIcons.pen :
FontAwesomeIcons.bookOpen,
      onPressed: () {
        newNoteController.readOnly = !readOnly;

        if (newNoteController.readOnly) {
          FocusScope.of(context).unfocus();
        } else {
          focusNode.requestFocus();
        }
      },
    ),
  ),
  Selector<NewNoteController, bool>(
    selector: (_, newNoteController) =>
newNoteController.canSaveNote,
    builder: (_, canSaveNote, __) => NoteIconButtonOutlined(
      icon: FontAwesomeIcons.check,
      onPressed: canSaveNote
        ? () {
            newNoteController.saveNote(context);
            Navigator.pop(context);
          }
        : null,
    ),
  ),
],
),
body: Padding(
  padding: const EdgeInsets.all(8.0),
  child: Column(
    children: [
      Selector<NewNoteController, bool>(
        selector: (context, controller) => controller.readOnly,
        builder: (context, readOnly, child) => TextField(
          controller: titleController,
          style: const TextStyle(
            fontSize: 24,
            fontWeight: FontWeight.bold,
          ),

```

```

        decoration: const InputDecoration(
          hintText: 'Title here',
          hintStyle: TextStyle(
            color: gray300,
          ),
          border: InputBorder.none,
        ),
        canRequestFocus: !readOnly,
        onChanged: (newValue) {
          newNoteController.title = newValue;
        },
      ),
    ),
    NoteMetadata(
      note: newNoteController.note,
    ),
    const Padding(
      padding: EdgeInsets.symmetric(vertical: 8.0),
      child: Divider(color: gray500, thickness: 2),
    ),
    Expanded(
      child: Selector<NewNoteController, bool>(
        selector: (_, controller) => controller.readOnly,
        builder: (_, readOnly, __) => Column(
          children: [
            Expanded(
              child: QuillEditor.basic(
                configurations: QuillEditorConfigurations(
                  controller: quillController,
                  placeholder: 'Note here...',
                  expands: true,
                  readOnly: readOnly,
                ),
              focusNode: focusNode,
            ),
            if (!readOnly) NoteToolbar(controller: quillController),
          ],
        ),
      ),
    ),
  ],
),
);
}
}

```

4.2.3. Код за контролер за нова бележка

```
import 'dart:convert';

import 'package:flutter/foundation.dart';
import 'package:flutter/material.dart';
import 'package:flutter_quill/flutter_quill.dart';
import 'package:provider/provider.dart';

import '../models/note.dart';
import 'notes_provider.dart';

class NewNoteController extends ChangeNotifier {
  Note? _note;
  set note(Note? value) {
    _note = value;
    _title = _note!.title ?? '';
    _content = Document.fromJson(jsonDecode(_note!.contentJson));
    _tags.addAll(_note!.tags ?? []);
    notifyListeners();
  }

  Note? get note => _note;

  bool _readOnly = false;
  set readOnly(bool value) {
    _readOnly = value;
    notifyListeners();
  }

  bool get readOnly => _readOnly;

  String _title = '';
  set title(String value) {
    _title = value;
    notifyListeners();
  }

  String get title => _title.trim();

  Document _content = Document();
  set content(Document value) {
    _content = value;
    notifyListeners();
  }
}
```

```

Document get content => _content;

final List<String> _tags = [];
void addTag(String tag) {
    _tags.add(tag);
    notifyListeners();
}

List<String> get tags => [..._tags];

void removeTag(int index) {
    _tags.removeAt(index);
    notifyListeners();
}

void updateTag(String tag, int index) {
    _tags[index] = tag;
    notifyListeners();
}

bool get isNewNote => _note == null;

bool get canSaveNote {
    final String? newTitle = title.isNotEmpty ? title : null;
    final String? newContent = content.toPlainText().trim().isEmpty
        ? content.toPlainText().trim()
        : null;

    bool canSave = newTitle != null || newContent != null;

    if (!isNewNote) {
        final newContentJson = jsonEncode(content.toDelta().toJson());
        canSave &= newTitle != note!.title ||
            newContentJson != note!.contentJson ||
            !listEquals(tags, note!.tags);
    }

    return canSave;
}

void saveNote(BuildContext context) {
    final String? newTitle = title.isNotEmpty ? title : null;
    final String? newContent = content.toPlainText().trim().isEmpty
        ? content.toPlainText().trim()
        : null;
    final String contentJson = jsonEncode(_content.toDelta().toJson());
    final int now = DateTime.now().microsecondsSinceEpoch;

```

```

    final Note note = Note(
      title: newTitle,
      content: newContent,
      contentJson: contentJson,
      dateCreated: isNewNote ? now : _note!.dateCreated,
      dateModified: now,
      tags: tags,
    );

    final notesProvider = context.read<NotesProvider>();
    isNewNote ? notesProvider.addNote(note) : notesProvider.updateNote(note);
  }
}

```

4.2.4. Код за страница за възстановяване на паролата

```

import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

import '../change_notifiers/registration_controller.dart';
import '../core/constants.dart';
import '../core/validator.dart';
import '../widgets/note_back_button.dart';
import '../widgets/note_button.dart';
import '../widgets/note_form_field.dart';

class RecoverPasswordpage extends StatefulWidget {
  const RecoverPasswordpage({super.key});

  @override
  State<RecoverPasswordpage> createState() => _RecoverPasswordpageState();
}

class _RecoverPasswordpageState extends State<RecoverPasswordpage> {
  late final TextEditingController emailController;

  GlobalKey<FormFieldState> emailKey = GlobalKey();

  @override
  void initState() {
    super.initState();

    emailController = TextEditingController();
  }

  @override
  void dispose() {
    emailController.dispose();
    super.dispose();
  }
}

```

```

}

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      leading: const NoteBackButton(),
      title: const Text('Recover Password'),
    ),
    body: Padding(
      padding: const EdgeInsets.all(16.0),
      child: Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          crossAxisAlignment: CrossAxisAlignment.stretch,
          children: [
            const Text(
              'Don\'t worry! Happens to the best of us!',
              textAlign: TextAlign.center,
            ),
            const SizedBox(height: 24),
            NoteFormField(
              key: emailKey,
              controller: emailController,
              fillColor: white,
              filled: true,
              labelText: 'Email',
              validator: Validator.emailValidator,
            ),
            const SizedBox(height: 24),
            SizedBox(
              height: 48,
              child: Selector<RegistrationController, bool>(
                selector: (_, controller) => controller.isLoading,
                builder: (_, isLoading, __) => NoteButton(
                  onPressed: isLoading
                    ? null
                    : () {
                        if (emailKey.currentState?.validate() ?? false) {
                          context
                            .read<RegistrationController>()
                            .resetPassword(
                              context: context,
                              email: emailController.text.trim(),
                            );
                        }
                      },
                  child: isLoading

```

```

        ? const SizedBox(
            width: 24,
            height: 24,
            child: CircularProgressIndicator(color: white),
        )
        : const Text('Send me a recovery link!'),
    ),
),
],
),
),
),
),
);
}
}

```

Заклучение и постигнати цели

В заключение, поставените в началото на дипломната разработка цели са постигнати. По време на работата бяха анализирани други платформи и сродни области, което допринесе за успешното изпълнение на задачите и за усъвършенстване на крайния продукт. Същевременно бяха очертани и важни насоки за бъдещо развитие в разглежданата област.

Цел 1: Да бъде създаден завършен софтуерен продукт, който да представлява универсално средство за хора, стремящи се към по-добра организираност и повишена ефективност при изпълнение на ежедневните си задачи.

В изпълнение на тази цел бяха поставени няколко задачи, чиято реализация е отразена в таблицата по-долу:

Задача	Анализ на резултата
Да се осигури интуитивен интерфейс, гарантиращ лека и бърза навигация в приложението;	Приложението включва интуитивен дизайн, който позволява бързо ориентиране и лесна навигация в платформата. То е разгледано в глава трета, точка 3.2.
Да се предостави възможност за изграждане на потребителски профил и вписване в системата;	В глава трета, подточка 3.2.1 и подточка 3.2.2 се проследява цялостният процес на разработка и комбинираните технологии, за да се постигне крайният резултат – приложение за бележки „Flutter Notes“, включващ пълен потребителски профил за взаимодействие със системата.
Да се реализира функционалност за добавяне на бележки, както и за прилагане на допълнителни опции и стилове;	Потребителят има възможност да създава бележки, което е описано в глава трета, подточка 3.2.7.
Да се създадат повече механизми за сортиране по дата на последна промяна и дата на създаване, както и опция за промяна на визуалния изглед на бележките.	В глава трета, подточка 3.2 и подточка 3.3 е подробно описана тази функционалност.

Таблица 1. *Постигнати резултати от Цел 1*

Извод: Въз основа на извършения анализ може да се заключи, че всички предварително формулирани задачи са изпълнени. В дипломната разработка са включени доказателства под формата на теоретично и практическо изложение, което съдържа множество примери и обосновани аргументи.

Цел 2: Да се осъществи анализ и сравнение на вече съществуващи платформи.

В изпълнение на тази цел бяха поставени няколко задачи, чиято реализация е отразена в таблицата по-долу:

Задача	Анализ на резултата
Да бъдат проучени най-популярните платформи, свързани с разглежданата проблематика;	В глава първа са описани най-често използваните платформи от подобен тип на територията на Интернет, свързани с темата на дипломната работа.
Да се анализира достъпността на тези платформи, включваща преглед на техния дизайн и структура;	На базата на следните ключови характеристики: • Целева група • Графичен потребителски интерфейс (дизайн) • Навигация (достъпност на приложението) • Съдържание (важност и структурираност на информацията) • Потребителска оценка е извършен анализ, насочен към подобряване на потребителското изживяване и удовлетворяване на неговите нужди.
Да се акцентира върху разнообразните функционалности	В първата глава се разглеждат най-често използваните приложения в

на платформите и начина, по който те се различават от останалите.	Интернет и техните ключови характеристики.
---	--

Таблица 2. Постигнати резултати от Цел 2

Извод: Проучването и анализът на съществуващите платформи от подобен тип, както и изследването на техните основни характеристики, допринесоха за прецизното дефиниране и усъвършенстване на концепцията на мобилното приложение за бележки – Flutter Notes.

Цел 3: Да бъде изграден краен софтуерен продукт, който да удовлетворява поставените цели, свързани с добавяне на записки, осигуряване на стабилна функционалност на потребителя на апликацията.

В изпълнение на тази цел бяха поставени няколко задачи, чиято реализация е отразена в таблицата по-долу:

Задача	Анализ на резултата
Да бъде разработена система с добра функционалност на потребителя, която да осигурява актуалност на данните, лесна организираност и бързи реакции;	В глава трета, подточка 3.1 са описани потребителските роли в мобилното приложение, както и техните нива на достъп и възможности за действия.
Да се проектира система за въвеждане на записки с множество полезни функции за потребителите в реално време, която да гарантира актуалност на данните и лесна организираност;	В глава трета, подточка 3.3 се разглеждат множеството полезни функции за потребителите в реално време, които гарантират актуалност на данните и лесна организираност.
Потребителят да може да добавя бележки, да ги изтрива, да ги редактира и да разполага с достъп до всички налични опции, които са вградени в приложението.	В глава трета, подточка 3.2 и подточка 3.3 е разгледана технологичната реализация на функционалността за добавяне, изтриване и редактиране бележки, и тяхното подробно функционално описание.

Таблица 3. Постигнати резултати от Цел 3

Извод: След реализирането на поставените цели, потребителите получиха възможност да извършват динамични действия, свързани с основната функционалност на мобилното приложение.

Цел 4: При разработката на мобилната платформа да бъдат използвани съвременни методи и технологии за мобилно програмиране.

В изпълнение на тази цел бяха поставени няколко задачи, чиято реализация е отразена в таблицата по-долу:

Задача	Анализ на резултата
Да бъдат представени технологиите, приложени при изграждането на конкретния софтуерен продукт;	В глава втора са разгледани основните технологии, използвани за реализацията на приложението.
Да се предоставят подробни сведения относно различните компоненти и използвания програмен код.	В глава четвърта, подточка 4.2, са представени програмните кодове с обяснения на някои от основните градивни елементи на мобилното приложение.

Таблица 4. *Постигнати резултати от Цел 4*

Извод: Въз основа на описания технологичен стек за програмиране беше разработено мобилното приложение за бележки – Flutter Notes.

Перспективи за бъдещо развитие

Разработеното мобилно приложение е замислено като основа за по-мощен бъдещ проект. Flutter Notes – Мобилното приложение за по-добра организираност чрез бележки, предлага множество възможности за иновации и разширяване на съществуващите функционалности.

Сред основните предвидени за бъдеща реализация насоки са:

- Създаване на версия за iOS с цел осигуряване на по-широка достъпност до услугите на приложението;
- Офлайн режим с последваща синхронизация;
- Интеграция с външни платформи и услуги;
- Генериране на статистически данни относно активността на отделните потребители.

Използвана литература

- [1] **Flutter**, Уикипедия - [https://en.wikipedia.org/wiki/Flutter_\(software\)](https://en.wikipedia.org/wiki/Flutter_(software)) [Онлайн] [Цитирано: 15.08.2026г.]
- [2] **Dart**, Уикипедия - [https://en.wikipedia.org/wiki/Dart_\(programming_language\)](https://en.wikipedia.org/wiki/Dart_(programming_language)) [Онлайн] [Цитирано: 15.08.2026г.]
- [3] **Firebase**, Уикипедия - <https://en.wikipedia.org/wiki/Firebase> [Онлайн] [Цитирано: 15.08.2026г.]
- [4] **Firestore - Documentation** - <https://firebase.google.com/docs/firestore> [Онлайн] [Цитирано: 15.08.2026г.]
- [5] **База данни – NoSQL** - <https://bg.wikipedia.org/wiki/NoSQL> [Онлайн] [Цитирано: 15.08.2026г.]
- [6] **Firebase for Flutter – Documentation** - <https://firebase.google.com/docs/flutter/setup?continue=https%3A%2F%2Ffirebase.google.com%2Flearn%2Fpathways%2Ffirebase-flutter%2523article-https%3A%2F%2Ffirebase.google.com%2Fdocs%2Fflutter%2Fsetup> [Онлайн] [Цитирано: 15.08.2026г.]
- [7] **Firebase CLI Reference - Documentation** - <https://firebase.google.com/docs/cli#mac-linux-npm> [Онлайн] [Цитирано: 15.08.2026г.]
- [8] **Get started with Firebase Authentication on Flutter – Documentation** - <https://firebase.google.com/docs/auth/flutter/start> [Онлайн] [Цитирано: 15.08.2026г.]
- [9] **Authenticate with Firebase using password-based accounts on Flutter – Documentation** - <https://firebase.google.com/docs/auth/flutter/password-auth> [Онлайн] [Цитирано: 15.08.2026г.]

- [10] **Client authentication – Documentation -**
<https://developers.google.com/android/guides/client-auth> [Онлайн]
[Цитирано: 15.08.2026г.]
- [11] **Силата на цветовете в дизайна на мобилни приложения -**
<https://www.whiz.bg/blog/silata-na-tsvetovete-v-dizayna-na-mobilni-prilozheniya.html> [Онлайн] [Цитирано: 15.08.2026г.]
- [12] **Cloudflare Workers AI -**
<https://developers.cloudflare.com/workers-ai/> [Онлайн] [Цитирано:
10.09.2026г.]